

Key-Insulated and Privacy-Preserving Signature Scheme with Publicly Derived Public Key, Revisited: Consistency, Outsider Strong Unforgeability, and Generic Construction

Keita Emura^{*1,2}

¹Kanazawa University, Japan.

²AIST, Japan.

September 3, 2025

Abstract

Liu et al. (EuroS&P 2019) introduced Key-Insulated and Privacy-Preserving Signature Scheme with Publicly Derived Public Key (PDPKS) to enhance the security of stealth address and deterministic wallet. In this paper, we point out that the current security notions are insufficient in practice, and introduce a new security notion which we call consistency. Moreover, we explore the unforgeability to provide strong unforgeability for outsider which captures the situation that nobody, except the payer and the payee, can produce a valid signature. From the viewpoint of cryptocurrency functionality, it allows us to implement a refund functionality. Currently, basically there is no way to refund a coin when one mistakenly spends a coin to an address. This functionality rescues the case, even in the stealth environment that hides information of the payer. Note that the refund functionality only works before the payee transfers a coin to own wallet, and it prevents a double spending issue. Finally, we propose a generic construction of PDPKS that provides consistency and outsider strong unforgeability. The design is conceptually much simpler than known PDPKS constructions. It is particularly note that the underlying strongly unforgeable signature scheme is required to provide the strong conservative exclusive ownership (S-CEO) security (Cremers et al., IEEE S&P 2021). Since we explicitly require the underlying signature scheme to be S-CEO secure, our security proof introduces a new insight of exclusive ownership security which may be of independent interest.

1 Introduction

1.1 Background

Liu et al. [37]¹ introduced Key-Insulated and Privacy-Preserving Signature Scheme with Publicly Derived Public Key (PDPKS) to capture and improve the functionality, security, and privacy requirements of stealth address [19, 46] and deterministic wallet [4, 24]. Briefly, the flow of PDPKS is described as follows (See Fig. 1). Assume that a payer Alice wants to transfer funds to a payee Bob. Bob publishes the master public key mpk_B where $(\text{mpk}_B, \text{msk}_B) \leftarrow \text{MasterKeyGen}(\text{pp})$ and $\text{pp} \leftarrow \text{PDPKS.Setup}(1^\lambda)$ is a common public parameter (here λ is a security parameter). Alice

^{*}k-emura@se.kanazawa-u.ac.jp

¹The full version of [37] is available in [38].

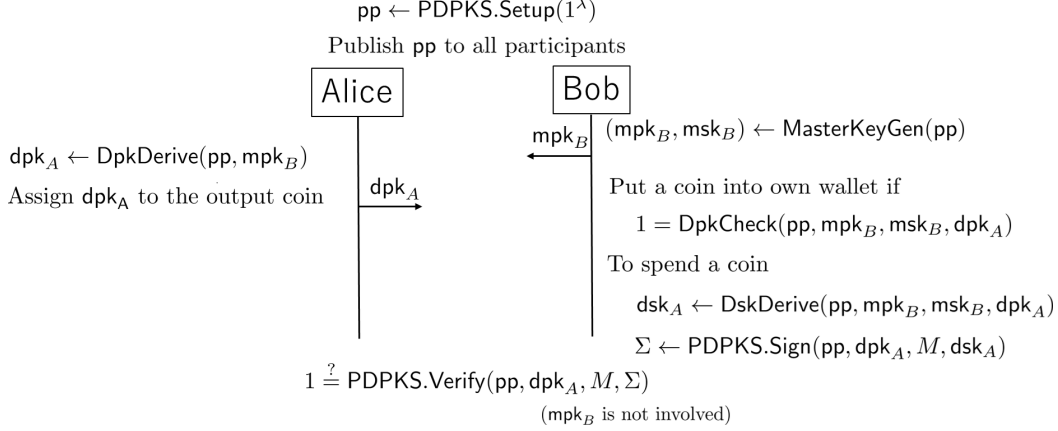


Figure 1: PDPKS Flow

derives a fresh public key $\text{dpk}_A \leftarrow \text{DpkDerive}(pp, \text{mpk}_B)$, assigns a coin to dpk_A , and sends dpk_A to Bob. We remark that this procedure is non-interactive, i.e., Alice can derive dpk_A from mpk_B without communicating with Bob. To spend a coin, Bob checks whether dpk_A was derived from mpk_B or not by running $\text{DpkCheck}(pp, \text{mpk}_B, \text{msk}_B, \text{dpk}_A)$. If the algorithm returns 1 (meaning that dpk_A is linked to mpk_B), then Bob generates the corresponding derived secret key $\text{dsk}_A \leftarrow \text{DskDerive}(pp, \text{mpk}_B, \text{msk}_B, \text{dpk}_A)$. We remark that this procedure is also non-interactive, i.e., Bob can derive dsk_A from dpk_A without communicating with Alice (after Bob obtains dpk_A). Bob generates a signature Σ on a message (transaction) M by running $\Sigma \leftarrow \text{PDPKS.Sign}(pp, \text{dpk}_A, M, \text{dsk}_A)$. Here, anyone can check the validity of (M, Σ) under dpk_A by running $\text{PDPKS.Verify}(pp, \text{dpk}_A, M, \Sigma)$ without using the corresponding master public key mpk_B .

Liu et al. [37] formalized security notions, correctness, unforgeability, and unlinkability. Correctness requires that honestly generated dpk and Σ are always accepted by DpkCheck and PDPKS.Verify algorithms, respectively. Unforgeability guarantees that no adversary $\mathcal{A}$ can produce $(\text{dpk}^*, M^*, \Sigma^*)$ where $\text{PDPKS.Verify}(pp, \text{dpk}^*, M^*, \Sigma^*) = 1$. Since dpk^* is produced by $\mathcal{A}$, it guarantees that even if anyone can derive a public key from a master public key mpk , nobody, except the corresponding master secret key holder, can produce a valid signature under the derived public key. Unlinkability guarantees that: (1) a derived public key does not leak information of the corresponding master public key, and (2) derived public keys do not leak information of whether two public keys are derived from the same master public key or not.

Liu et al. [37] pointed out that identity-based signature (IBS) is a promising tool to construct PDPKS but required a special property, that they called MPK-pack-able property. They found that the Barreto et al. IBS scheme [7] has the property, and proposed a pairing-based PDPKS scheme which is secure under a q -type assumption in the random oracle model. We introduce their construction methodology for more detail in the Appendix. As a subsequent work, Liu et al. [35] proposed a lattice-based PDPKS scheme by combining a lattice basis delegation technique [3] and public key encryption (PKE) with key privacy [10]. As a concrete lattice-based instantiation of the underlying key private PKE scheme, they proposed a lattice-based key private PKE scheme based on the Regev PKE scheme [43] with the Fujisaki-Okamoto transformation [27] to prevent chosen-ciphertext attacks (CCA).

1.2 Our Motivation

Generic Construction. Though the first construction [37] is based on IBS, it is not a purely generic construction (Liu et al. mentioned that “*We propose a (partially) generic approach on how to obtain a PDPKS construction*”). Moreover, no generic transformation for adding the MPK-packable property to an IBS scheme has not been proposed so far. The second construction [35] relies on the specific lattice basis delegation technique (though any key private PKE can be employed). Giving a generic construction highlights what a sufficient condition is to construct a cryptographic primitive, and provides several instantiations. Thus, proposing a generic construction of PDPKS is still open and is desirable.

Security Definitions. We also revisited the security definitions.

- First, we point out that the security notions defined in [35, 37] are not sufficient in practice because no security of the `DpkCheck` algorithm is defined. Due to the usage of PDPKS given in Fig. 1, a payer Alice uses dpk_A to specify the receiver of the transaction, and a payee Bob puts a coin into own wallet if $\text{DpkCheck}(\text{pp}, \text{mpk}_B, \text{msk}_B, \text{dpk}_A) = 1$. That is, Bob verifies whether dpk_A is linked to mpk_B or not before spending a coin. We remark that correctness just guarantees that $\text{DpkCheck}(\text{pp}, \text{mpk}_B, \text{msk}_B, \text{dpk}_A) = 1$ holds for a derived public key $\text{dpk}_A \leftarrow \text{DpkDerive}(\text{pp}, \text{mpk}_B)$, and does not guarantee that $\text{DpkCheck}(\text{pp}, \text{mpk}_B, \text{msk}_B, \text{dpk}_A) = 0$ if dpk_A is not derived from mpk_B . We need to strengthen the security of PDPKS in this perspective.
- Second, we explore unforgeability. In the current definition, an adversary $\mathcal{A}$ declares the challenge derived public key dpk^* which captures the situation that nobody, except the master secret key holder (the payee), can produce a valid signature, namely, the payer also is not allowed to produce a valid signature. In the actual usage, however, it seems acceptable that the payer also can produce a valid signature unless a third person who just observes dpk cannot produce a valid signature. From this perspective, we can define a weaker variant of unforgeability, which we call outsider unforgeability, where dpk^* is sent to $\mathcal{A}$ from the challenger which captures the situation that nobody, except the payer and the payee, can produce a valid signature. This relaxation allows us to avoid the above IBS-like construction. From the viewpoint of cryptocurrency functionality, it allows us to implement a refund functionality, and it can be used for instantiating a refundable stealth address. Currently, basically there is no way to refund a coin when one mistakenly spends a coin to an address. This functionality rescues the case, even in the stealth environment that hides information of the payer. Note that the refund functionality only works before the payee transfers a coin to own wallet, and it prevents a double spending issue. Later, we will mention that refundability has been implicitly realized via adaptor signatures.
- Third, we further explore unforgeability. The original definition captures a conventional existential unforgeability: it is required that an adversary $\mathcal{A}$ never sends a signing query (dpk^*, M^*) when $\mathcal{A}$ outputs a forgery $(\text{dpk}^*, M^*, \Sigma^*)$. This means that a signature might be re-randomizable by definition. That is, without contradicting unforgeability, anyone may be able to produce a valid message-signature pair (M^*, Σ) by re-randomizing Σ^* . In the cryptocurrency context, a third person (who is neither the payer nor the payee) may be able to produce a valid (M^*, Σ) on an existing transaction M^* . Of course, if an adversary generates multiple signatures on an already signed transaction, it cannot forge transactions or mount double spend attacks. However, any situation that a third party (who does not

have the signing key) may produce a valid signature should be avoided as much as possible. Thus, we consider strong unforgeability: $\mathcal{A}$ is allowed to issue (dpk^*, M) as a signing query where $M = M^*$ is allowed, and obtains Σ . For the forgery $(\text{dpk}^*, M^*, \Sigma^*)$, it is required that $(M^*, \Sigma^*) \notin \{(M, \Sigma)\}$ holds.

1.3 Our Contribution

In this paper, we revisited the security notions of PDPKS. First, we introduce a new security notion which we call consistency: for (distinct) two master public keys mpk_0 and mpk_1 , it guarantees that $\text{DpkCheck}(\text{pp}, \text{mpk}_0, \text{msk}_0, \text{dpk}) = 0$ when $\text{dpk} \leftarrow \text{DpkDerive}(\text{pp}, \text{mpk}_1)$ with overwhelming probability (we define consistency in a computational security manner later). We clarify that if a PDPKS scheme does not provide consistency, then the meaningless construction, the DpkCheck algorithm always outputs 1 regardless of the input, does not affect correctness. Then, Bob cannot check whether dpk_A is incorrect or not via the DpkCheck algorithm.

Second, we weaken unforgeability which we call outsider unforgeability. Third, we strengthen unforgeability to capture strong unforgeability. By combining outsider unforgeability, which we call outsider strong unforgeability, our definition guarantees the situation that nobody, except the payer and the payee, can produce a valid signature.

As mentioned in the motivation part, from the viewpoint of functionality, outsider unforgeability allows us to implement a refund functionality. Moreover, it also allows us to remove the IBS-like construction procedure unlike the Liu et al.’s construction, and can provide a generic construction of PDPKS. In our construction, a payer Alice chooses a verification and signing key pair of a signature scheme $(\text{vk}_A, \text{sigk}_A)$ on the fly, and encrypts sigk_A by using the payee’s public key $\text{mpk}_B = \text{PKE.pk}_B$. That is, nobody, except Alice and the decryption key holder Bob, can produce the corresponding signing key sigk_A which means that vk_A is linked to mpk_B . By using sigk_A , Bob can produce a signature which is valid under vk_A . The design is conceptually much simpler than known PDPKS constructions because the ingredients are only signatures and PKE, and thus we can easily provide strong unforgeability. The proposed generic construction provides correctness, outsider strong unforgeability, unlinkability, and consistency.

Feasibility of the Generic Construction. The underlying signature scheme is assumed to be strongly unforgeable. As a candidate, we can employ the Boneh-Shen-Waters signature scheme [13] which is strongly unforgeable under the computational Diffie-Hellman assumption in bilinear groups, and is secure in the standard model. We can also employ generic conversions [44, 45] to obtain a strongly unforgeable signature scheme from any signature scheme. We also require that the signature scheme provides strong conservative exclusive ownership (S-CEO) security [21]. S-CEO is recognized as one of BUFF (Beyond UnForgeability Features) security, and it informally guarantees that for a verification key vk^* , no adversary can produce vk such that there is (M, Σ) satisfying $\text{Sig.Verify}(\text{vk}^*, M, \sigma) = 1$, $\text{Sig.Verify}(\text{vk}, M, \sigma) = 1$, and $\text{vk} \neq \text{vk}^*$. Cremers et al. [21] gave a generic transformation to provide the S-CEO security by adding $\text{Hash}(\text{vk}, M)$ to a signature. Here, Hash is a collision resistant hash function and is *not* modeled as a random oracle. We can add the S-CEO security to the Boneh-Shen-Waters signature scheme via the generic transformation. As a concrete scheme, Brendel et al. [14] showed that the Ed25519-LibS signature scheme is strong unforgeable and provides the malicious-strong universal exclusive ownership (M-S-UEO) security that implies the S-CEO security. Moreover, Aulbach et al. [5] showed that NIST PQC candidates, CROSS²,

²<https://www.cross-crypto.com/>

Table 1: Comparisons of PDPKS Schemes

Scheme	Assumptions	ROM/SDM	Unforgeability
Liu et al. [37]	q -SDH, CDH (in bilinear groups)	ROM	Normal
Liu et al. [35]	LWE, SIS	ROM	Normal
Ours 1	CDH (in bilinear groups), DDH, CR	SDM	Strong and Outsider
Ours 2	ECDLP, ODH	ROM	Strong and Outsider
Ours 3	ECDLP, DDH, CR	ROM	Strong and Outsider
Ours 4-1	MLWE, SIS, CR, LWE, GapSVP	ROM	Strong and Outsider
Ours 4-2	One-more SVP, CR, LWE, GapSVP	ROM	Strong and Outsider
Ours 4-3	Self-target MSIS, CR, LWE, GapSVP	ROM	Strong and Outsider

Table 2: Efficiency Comparisons of DL/Pairing-based PDPKS Schemes

Scheme	mpk	msk	dpk	dsk	\Sigma
Liu et al. [37]	$2 \mathbb{G}_2 $	$2 \mathbb{Z}_p $	$2 \mathbb{G}_2 $	$ \mathbb{G}_1 $	$ \mathbb{G}_1 + \mathbb{Z}_p $
Generic Construction	PKE.pk	PKE.dk	$ \mathbf{vk} + C_{\text{PKE}} $	sigk	\Sigma
Ours 1	$3 \mathbb{G} $	$6 \mathbb{Z}_p $	$O(\log \lambda) \mathbb{G}_1 + 4 \mathbb{G} $	$ \mathbb{G}_1 $	$ \mathbb{G}_1 + \mathbb{G}_2 + \mathbb{Z}_p + \text{CR} $
Ours 2	$ \mathbb{G} $	$ \mathbb{Z}_p $	$2 \mathbb{G} + C_{\text{SKE}} + \text{MAC} $	$ \mathbb{Z}_p $	$ \mathbb{G} + \mathbb{Z}_p $
Ours 3	$3 \mathbb{G} $	$6 \mathbb{Z}_p $	$5 \mathbb{G} $	$ \mathbb{Z}_p $	$ \mathbb{G} + \mathbb{Z}_p $

HAETA³, HAWK⁴, RACCOON⁵, and PROV⁶, provide the S-CEO security. Among them, we can employ HAETA, HAWK, and RACCOON because they are strongly unforgeable. See the algorithm specification documents⁷ for details.

The underlying CCA-secure PKE scheme is assumed to be key private [10] (it ensures that a ciphertext does not leak information of the public key) and strongly robust [1] (it ensures that the decryption result of a ciphertext is $\perp$ if the ciphertext is not produced by the corresponding public key). Abdalla et al. [1] gave a strongly robust variant of the Cramer-Shoup PKE scheme [20] and the DHIES (Diffie-Hellman integrated encryption scheme) PKE scheme [2], respectively, that are also CCA secure and key private. Abdalla et al. [1] also proposed a generic transformation to add robustness to any key private and CCA-secure PKE scheme. The transformation additionally requires a commitment scheme that provides standard hiding and binding properties. Thus, for a lattice-based instantiation of the PKE scheme, we can employ a key private variant of the Regev PKE given by Liu et al. [35], with the Abdalla et al. transformation. Finally, we emphasize that the proposed generic construction itself does not rely on random oracles. Thus, instantiated PDPKS schemes are secure in the (quantum) random oracle model if the building block relies on (quantum) random oracles.

Comparison. In summary, we compare interesting instantiations with previous schemes in Table 1. Here, ROM stands for random oracle model and SDM stands for standard model. SDH stands for strong Diffie-Hellman, CDH/DDH stands for computational/decisional Diffie-Hellman, (M)LWE stands for (module) learning with errors, SIS stands for short integer solution, CR stands for collision resistant hash, ODH stands for oracle Diffie-Hellman, and SVP stands for shortest vector problem.

Ours 1: A pairing-based PDPKS scheme without random oracles instantiated from the Boneh-Shen-Waters signature scheme [13] with Cremers et al.’s conversion [21] and a strongly robust

³<https://kpmc.cryptolab.co.kr/haetae>

⁴<https://hawk-sign.info/>

⁵<https://raccoonfamily.org/>

⁶<https://prov-sign.github.io/>

⁷<https://csrc.nist.gov/projects/pqc-dig-sig>

variant of the Cramer-Shoup PKE scheme [1]. Since previous PDPKS schemes are secure in the random oracle model. Due to the result by Canetti et al. [15], random oracles should not be employed as much as possible.

Ours 2: A discrete-logarithm (DL) based pairing-free PDPKS scheme in the random oracle model instantiated from the Ed25519-LibS signature scheme [14] and a strongly robust variant of the DHIES PKE scheme [1].

Ours 3: A DL based pairing-free PDPKS scheme in the random oracle model instantiated from the Ed25519-LibS signature scheme [14] and a strongly robust variant of the Cramer-Shoup PKE scheme [1]. Compared to the scheme described as “Ours 2”, we can avoid to employ an oracle assumption, and the scheme is secure under standard assumptions in the random oracle model, whereas the Liu et al. scheme [37] relies on the q -SDH assumption. Due to the Cheon attack [17], q -type assumptions should not be employed as much as possible.

Ours 4: Lattice-based PDPKS schemes in the random oracle model instantiated from (4-1) HAETAE, (4-2) HAWK or (4-3) RACCOON, and a strongly robust variant of the Liu et al.’s key private PKE scheme (i.e., the key private PKE scheme given in [35] is converted by the Abdalla et al.’s transformation by using a lattice-based commitment scheme such as the KXT commitment [33]).

We also compare the size of keys and signatures among DL/pairing-based PDPKS schemes in Table 2. To clarify these sizes in the proposed generic construction, we add the generic construction row in the table. In the generic construction, a PDPKS signature is a signature of the underlying signature scheme. Thus, we use the same notation Σ . Let $(\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T)$ be bilinear groups with prime order p and $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be a pairing (i.e., BN curves [9] or BLS curves [8]). Let $\mathbb{G}$ be a group with prime order p (i.e., Curve25519 [11]). $|\text{CR}|$ is due to Cremers et al.’s conversion, and is estimated as 2λ due to the birthday bound. For Ed25519, a signature contains an element of the elliptic curve and a value modulo L . Here, L is specified by the prime generator order where for a generator $g \in \mathbb{G}$, $g^L = 1$ holds.⁸ Thus, we denote $|\mathbb{G}| + |\mathbb{Z}_p|$ in the table as the signature size of Ed25519.

Remark. Kulkarni and Xagawa [34] showed that several MPC-in-the-head signatures are strongly unforgeable and provide the S-CEO security. These signature schemes also can be building blocks of our generic construction. For example, from the candidates in Round 2 of the NIST additional PQC signature standardization, we can employ MQOM⁹, PERK¹⁰, RYDE¹¹, and SDitH¹². However, we cannot specify the corresponding PKE scheme secure under the same/similar complexity assumptions of those of these signatures. Thus, we do not employ them in Table 1.

Technical Overview. We further explore the reason why an IBS-like construction is required in the Liu et al. PDPKS scheme [37]. Due to the syntax of PDPKS, anyone who observes a master public key mpk_B can derive a public key dpk_A , and unforgeability guarantees that nobody, except the corresponding master secret key holder (the payee Bob), can produce a valid signature under the derived public key dpk_A . That is, dsk_A is derived from dpk_A by using msk_B after dpk_A is generated. This key generation process requires an IBS-like construction procedure. Here, we

⁸The authors of [14] employ additive operations. We introduce their notations: E is an elliptic curve, and B is a generator of the prime order subgroup of E , and c is the $\log_2$ of curve cofactor, Then, $LB = 0$ and $2^c L = |E|$ hold.

⁹<https://mqom.org/>

¹⁰<https://pqc-perk.org>

¹¹<https://pqc-ryde.org/>

¹²<https://sdith.org/>

weaken the unforgeability that captures the situation that nobody, except the payer and the payee, can produce a valid signature. This relaxation allows a payer Alice to produce dsk_A together with dpk_A and can remove the IBS-like construction procedure. In our construction, a payer Alice chooses a verification and signing key pair of a signature scheme $(\text{vk}_A, \text{sigk}_A)$ on the fly, and encrypts sigk_A by using the payee's public key $\text{mpk}_B = \text{PKE.pk}_B$. That is, nobody, except Alice and the decryption key holder Bob, can produce the corresponding signing key sigk_A which means that vk_A is linked to mpk_B . By using sigk_A , Bob can produce a signature which is valid under vk_A . The design is conceptually much simpler than known PDPKS constructions.

We need to further assume that the underlying signature scheme is S-CEO secure, which is the most technical part of this paper. We give an intuition of the security proof of outsider strong unforgeability (See Section 5 for detail). Let $\text{dpk}^* = (\text{vk}^*, C_{\text{PKE}}^*)$ where C_{PKE}^* is a ciphertext of sigk^* . Before reducing strong unforgeability of the underlying signature scheme, we need to guarantee that $\mathcal{A}$ does not produce dpk as a derived secret key corruption query such that $\text{dpk} = (\text{vk}, C_{\text{PKE}}^*)$, $\text{vk} \neq \text{vk}^*$, but vk is a valid verification key relative to sigk^* . Since the DpkCheck algorithm returns 1 for dpk , the reduction algorithm needs to respond sigk^* and fails to reduce strong unforgeability. To exclude the case, we employ the S-CEO security. The reduction algorithm takes vk^* from the challenger of the S-CEO security. C_{PKE}^* has been replaced by a ciphertext of $0^{|\text{sigk}|}$ by employing the IND-CCA security of the PKE scheme. Thus, the reduction algorithm can produce $\text{dpk}^* = (\text{vk}^*, C_{\text{PKE}}^*)$. If an adversary sends $\text{dpk} = (\text{vk}, C_{\text{PKE}}^*)$ where $\text{vk} \neq \text{vk}^*$, the reduction algorithm sends a signing query m to the challenger, and obtains a signature Σ . If $\text{Sig.Verify}(\text{vk}, m, \Sigma) = 1$, then the reduction algorithm breaks the S-CEO security, and does not have to respond the derived secret key corruption query for dpk to reduce strong unforgeability.

Related Work. As a similar primitive of PDPKS, Wang et al. [47] proposed key derivable signature (KDS) that allows a master secret key to sign a message unlike PDPKS, and proposed a generic construction of KDS from a key derivation scheme (KDV), PKE, and signatures *in the random oracle model*. Though their construction methodology could be employed to construct PDPKS, it deeply depends on random oracles, whereas our generic construction of PDPKS itself does not rely on random oracles. Furthermore, we clarify that KDS does not support refundability. This highlights that their generic construction of KDS does not imply our proposed generic construction. Pu et al. [42] proposed a post-quantum stealth signature scheme which they call Spirit. They also proposed a generic transformation from a stealth signature scheme without key-exposure into a scheme with unbounded key-exposure. Their construction is not purely generic in the sense that they first constructed a stealth signature scheme based on Dilithium and transformed it to a scheme with key exposure. As an independent and concurrent work, Mongardini et al. [40] proposed identity-based matchmaking signatures (IB-MSS) based on the stealth signatures modeled in [42]. As in our outsider unforgeability, the challenge one-time public key is generated by the challenger, and is not generated by the adversary. One crucial difference is IB-MSS involves a certification authority (CA) that generates sender/receiver keys using a master secret key to ensure that participants comply with specific regulations, such as anti-money laundering (AML) and know your customer (KYC) requirements.

Related Work in terms of Exclusive Ownership security. We explicitly assume that the underlying signature scheme is S-CEO secure. Unlike strong unforgeability, which is widely used for enhancing a security level, e.g., the CHK transformation [16], exclusive ownership security notions have not been widely employed as a security property of the underlying signature scheme, with the following one exception (to the best of our knowledge). Gunther et al. [31] showed that a lightweight authenticated key exchange protocol for IoT communication, EDHOC (Ephemeral Diffie-Hellman Over COSE, COSE stands for CBOR Object Signing and Encryption, and CBOR

stands for Concise Binary Object Representation), the underlying signature scheme is explicitly assumed to be universal exclusive ownership (S-UEO) secure to provide multi-stage key exchange security.

As another related works, Boneh et al. [12] introduced strong binding for multi-signatures as a related definition of message-bound signatures. Ferreira and Pascal [25] employed Dilithium and EdDSA to instantiate their post-quantum secure ZRTP (which is an authenticated key exchange protocol for establishing secure communications for Voice over IP applications) and mentioned that both signature schemes are S-UEO secure. But they did not explicitly require the S-UEO security to prove the security of their protocol. Jiang and Wang [32] also mentioned exclusive ownership security notions, but they did not employ the notions and claimed that their protocol is secure under the standard unforgeability because both the user and servers employ the user's fixed public key for verification.

Since we explicitly require the underlying signature scheme to be S-CEO secure, our security proof introduces a new insight of exclusive ownership security which may be of independent interest.

Adaptor Signatures. We regard refundability is a suitable functionality, and leave further considerations on the functionality in detail as a future work. Nevertheless, we would like to mention that refundability has been implicitly realized via adaptor signatures, e.g., [6, 18, 22, 28, 36, 41]. When Alice has a token c_A for some cryptocurrency and Bob has a witness y of some instance Y , we consider the case that Alice and Bob would like to trade c_A and y . Alice generates a pair of verification key and signing key $(vk, sigk)$ and posts a transaction to the blockchain that transfers c_A to Bob if a valid signature σ under vk is sent. Then, Alice generates a pre-signature on the transaction using $sigk$ and Y , and sends it to Bob. Bob adopts the pre-signature and generates a full signature using y . Since the full signature is valid under vk , Bob can obtain c_A by sending σ to the blockchain. After σ is published, Alice can extract y from the pre-signature and the full signature. Thus, a fair exchange completes.

Obviously, Alice can withdraw c_A because Alice has $sigk$. Gokay et al. [29] explicitly employed the refund phase when their atomic swap protocol cannot be completed within the time parameter. As in outsider unforgeable PDPKS, the refund functionality only works before the payee transfers a coin to own wallet, and it prevents a double spending issue. One crucial difference from ours is that the adaptor signature based refundable address is not stealth. The common point is both Alice (the payer) and Bob (the payee) can issue the transaction to spend c_A . From this perspective, PDPKS and adaptor signatures may have a relationship.

2 Preliminaries

In this section, we define signatures and PKE. We introduce setup algorithms to employ the same parameter among all users (payers and payees).

2.1 Signatures

Let $\text{Sig} = (\text{Sig.Setup}, \text{Sig.KeyGen}, \text{Sig.Sign}, \text{Sig.Verify})$ be a signature scheme. The setup algorithm Sig.Setup takes a security parameter λ , and outputs a common parameter pp_{Sig} that implicitly contains a message space MS_{Sig} . The key generation algorithm takes pp_{Sig} as input and outputs a verification and signing key pair $(vk, sigk)$. The signing algorithm Sig.Sign takes $sigk$ and a message to be signed $M \in \text{MS}_{\text{Sig}}$ as input and outputs a signature Σ . The verification algorithm takes vk , M , and Σ as input and outputs 0 or 1. The correctness requires that for any $\lambda \in \mathbb{N}$, any $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(\text{pp}_{\text{Sig}})$, any $(vk, sigk) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$, and any $M \in \text{MS}_{\text{Sig}}$,

$\text{Sig.Verify}(\text{vk}, M, \text{Sig.Sign}(\text{sigk}, M)) = 1$ holds with overwhelming probability in the security parameter λ .

Strong EUF-CMA. Strong unforgeability is defined as follows. Let $\mathcal{A}$ be an adversary and $\mathcal{C}$ be the challenger. $\mathcal{C}$ generates $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(\text{pp}_{\text{Sig}})$ and $(\text{vk}, \text{sigk}) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$, and gives $(\text{pp}_{\text{Sig}}, \text{vk})$ to $\mathcal{A}$. $\mathcal{C}$ initiates $\text{SigSet} = \emptyset$. $\mathcal{A}$ is allowed to issue signing queries M . $\mathcal{C}$ generates $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}, M)$ and returns Σ to $\mathcal{A}$. $\mathcal{C}$ stores (M, Σ) to SigSet . Finally, $\mathcal{A}$ outputs (M^*, Σ^*) . We say that $\mathcal{A}$ wins if $\text{Sig.Verify}(\text{vk}, M^*, \Sigma^*) = 1$ and $(M^*, \Sigma^*) \notin \text{SigSet}$. The advantage of $\mathcal{A}$ is defined as $\text{Adv}_{\mathcal{A}, \text{Sig}}^{\text{strong}}(\lambda) = \Pr[\mathcal{A} \text{ wins}]$. We say that Sig is strongly unforgeable if $\text{Adv}_{\mathcal{A}, \text{Sig}}^{\text{strong}}(\lambda)$ is negligible for all probabilistic polynomial-time (PPT) adversaries $\mathcal{A}$.

S-CEO. The S-CEO security is defined as follows. Let $\mathcal{A}$ be an adversary and $\mathcal{C}$ be the challenger. $\mathcal{C}$ generates $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(\text{pp}_{\text{Sig}})$ and $(\text{vk}, \text{sigk}) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$, and gives $(\text{pp}_{\text{Sig}}, \text{vk})$ to $\mathcal{A}$. $\mathcal{C}$ initiates $\text{SigSet} = \emptyset$. $\mathcal{A}$ is allowed to issue signing queries M . $\mathcal{C}$ generates $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}, M)$ and returns Σ to $\mathcal{A}$. $\mathcal{C}$ stores (M, Σ) to SigSet . Finally, $\mathcal{A}$ outputs $(\text{vk}^*, M^*, \Sigma^*)$. We say that $\mathcal{A}$ wins if $\text{Sig.Verify}(\text{vk}^*, M^*, \Sigma^*) = 1$, $(M^*, \Sigma^*) \in \text{SigSet}$ (that implies $\text{Sig.Verify}(\text{vk}, M^*, \Sigma^*) = 1$), and $\text{vk}^* \neq \text{vk}$. The advantage of $\mathcal{A}$ is defined as $\text{Adv}_{\mathcal{A}, \text{Sig}}^{\text{S-CEO}}(\lambda) = \Pr[\mathcal{A} \text{ wins}]$. We say that Sig is S-CEO secure if $\text{Adv}_{\mathcal{A}, \text{Sig}}^{\text{S-CEO}}(\lambda)$ is negligible for all PPT adversaries $\mathcal{A}$.

2.2 PKE

Let $\text{PKE} = (\text{PKE.Setup}, \text{PKE.KeyGen}, \text{PKE.Enc}, \text{PKE.Dec})$ be a PKE scheme. The setup algorithm PKE.Setup takes a security parameter λ as input, and outputs a common parameter pp_{PKE} that implicitly contains a message space MS_{PKE} . The key generation algorithm PKE.KeyGen takes pp_{PKE} , and outputs a key pair $(\text{PKE.pk}, \text{PKE.dk})$. The encryption algorithm PKE.Enc takes PKE.pk and a plaintext M , and outputs a ciphertext C_{PKE} . The decryption algorithm PKE.Dec takes PKE.dk and C_{PKE} as input, and outputs M or $\perp$. Correctness requires that for any $\lambda \in \mathbb{N}$, any $\text{pp}_{\text{PKE}} \leftarrow \text{PKE.Setup}(1^\lambda)$, any $(\text{PKE.pk}, \text{PKE.dk}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, any $M \in \text{MS}_{\text{PKE}}$, $\text{PKE.Dec}(\text{PKE.dk}, \text{PKE.Enc}(\text{PKE.pk}, M)) = M$ holds with overwhelming probability in the security parameter λ .

CCA. The CCA security is defined as follows. Let $\mathcal{A}$ be an adversary and $\mathcal{C}$ be the challenger. $\mathcal{C}$ generates $\text{pp}_{\text{PKE}} \leftarrow \text{PKE.Setup}(1^\lambda)$ and $(\text{PKE.pk}, \text{PKE.dk}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, and gives $(\text{pp}_{\text{PKE}}, \text{PKE.pk})$ to $\mathcal{A}$. $\mathcal{A}$ is allowed to issue decryption queries C_{PKE} . $\mathcal{C}$ returns the result of $\text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})$. $\mathcal{A}$ declares two equal-length plaintexts M_0^* and M_1^* . $\mathcal{C}$ chooses $b \xleftarrow{\$} \{0, 1\}$, computes the challenge ciphertext $C_{\text{PKE}}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, M_b^*)$, and gives C_{PKE}^* to $\mathcal{A}$. $\mathcal{A}$ is further allowed to issue decryption queries $C_{\text{PKE}} \neq C_{\text{PKE}}^*$. $\mathcal{C}$ returns the result of $\text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})$. Finally, $\mathcal{A}$ outputs $b' \in \{0, 1\}$. $\mathcal{A}$ wins if $b = b'$. The advantage of $\mathcal{A}$ is defined as $\text{Adv}_{\mathcal{A}, \text{PKE}}^{\text{IND-CCA}}(\lambda) = |\Pr[b = b'] - 1/2|$. We say that PKE is IND-CCA secure (or simply CCA secure) if $\text{Adv}_{\mathcal{A}, \text{PKE}}^{\text{IND-CCA}}(\lambda)$ is negligible for all PPT adversaries $\mathcal{A}$.

Key Privacy. Key privacy is defined as follows. Note that the following definition contains CCA security. Let $\mathcal{A}$ be an adversary and $\mathcal{C}$ be the challenger. $\mathcal{C}$ generates $\text{pp}_{\text{PKE}} \leftarrow \text{PKE.Setup}(1^\lambda)$, $(\text{PKE.pk}_0, \text{PKE.dk}_0) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, and $(\text{PKE.pk}_1, \text{PKE.dk}_1) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, and gives $(\text{pp}_{\text{PKE}}, \text{PKE.pk}_0, \text{PKE.pk}_1)$ to $\mathcal{A}$. $\mathcal{A}$ is allowed to issue decryption queries (C_{PKE}, i) where $i \in \{0, 1\}$. $\mathcal{C}$ returns the result of $\text{PKE.Dec}(\text{PKE.dk}_i, C_{\text{PKE}})$. $\mathcal{A}$ declares the challenge plaintext M^* . $\mathcal{C}$ chooses $b \xleftarrow{\$} \{0, 1\}$, computes the challenge ciphertext $C_{\text{PKE}}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}_b, M^*)$, and gives C_{PKE}^* to $\mathcal{A}$. $\mathcal{A}$ is further allowed to issue decryption queries (C_{PKE}, i) where $i \in \{0, 1\}$ and $C_{\text{PKE}} \neq C_{\text{PKE}}^*$. $\mathcal{C}$ returns the result of $\text{PKE.Dec}(\text{PKE.dk}_i, C_{\text{PKE}})$. Finally, $\mathcal{A}$ outputs $b' \in \{0, 1\}$. $\mathcal{A}$

wins if $b = b'$. The advantage of $\mathcal{A}$ is defined as $\text{Adv}_{\mathcal{A}, \text{PKE}}^{\text{Key-Privacy}}(\lambda) = |\Pr[b = b'] - 1/2|$. We say that PKE is key private if $\text{Adv}_{\mathcal{A}, \text{PKE}}^{\text{Key-Privacy}}(\lambda)$ is negligible for all PPT adversaries $\mathcal{A}$.

Strong Robustness. Strong robustness is defined as follows (here strong means that robustness holds for ciphertexts declared by an adversary). Let $\mathcal{A}$ be an adversary and $\mathcal{C}$ be the challenger. $\mathcal{C}$ generates $\text{pp}_{\text{PKE}} \leftarrow \text{PKE.Setup}(1^\lambda)$, $(\text{PKE.pk}_0, \text{PKE.dk}_0) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, and $(\text{PKE.pk}_1, \text{PKE.dk}_1) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, and gives $(\text{pp}_{\text{PKE}}, \text{PKE.pk}_0, \text{PKE.pk}_1)$ to $\mathcal{A}$. $\mathcal{A}$ declares the challenge ciphertext C_{PKE}^* . $\mathcal{C}$ runs $M_0 \leftarrow \text{PKE.Dec}(\text{PKE.dk}_0, C_{\text{PKE}}^*)$ and $M_1 \leftarrow \text{PKE.Dec}(\text{PKE.dk}_1, C_{\text{PKE}}^*)$. $\mathcal{C}$ outputs 1 if $M_0 \neq \perp$ and $M_1 \neq \perp$ hold, and 0 otherwise. The advantage of $\mathcal{A}$ is defined as $\text{Adv}_{\mathcal{A}, \text{PKE}}^{\text{strong-robust}}(\lambda) = \Pr[\mathcal{C} \rightarrow 1]$. We say that PKE is strongly robust if $\text{Adv}_{\mathcal{A}, \text{PKE}}^{\text{strong-robust}}(\lambda)$ is negligible for all PPT adversaries $\mathcal{A}$.

2.3 PDPKS

Next, we introduce the syntax and security notions of PDPKS defined by Liu et al. [37]. We note that the definition of consistency and outsider unforgeability, that are newly introduced in this paper, are given in Section 3. Let PDPKS be a PDPKS scheme that consists of seven algorithms (PDPKS.Setup, MasterKeyGen, DpkDerive, DpkCheck, DskDerive, PDPKS.Sign, PDPKS.Verify) defined as follows.

PDPKS.Setup: The setup algorithm takes a security parameter λ as input, and outputs a common parameter pp that implicitly contains a message space MS_{PDPKS} .

MasterKeyGen: The master key generation algorithm takes pp as input, and outputs a master public key and a master secret key pair (mpk, msk) .

DpkDerive: The public key derivation algorithm takes pp and mpk as input, and outputs a derived public key dpk .

DpkCheck: The derived public key checking algorithm takes pp , mpk , msk , and dpk as input, and outputs 1 (meaning that dpk is linked to mpk) or 0 (meaning that dpk is not linked to mpk).

DskDerive: The secret key derivation algorithm takes pp , mpk , msk , and dpk as input. The algorithm outputs $\perp$ if $\text{DpkCheck}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk}) = 0$. Otherwise, the algorithm outputs a derived secret key dsk corresponding to dpk .

PDPKS.Sign: The signing algorithm takes pp , dpk , dsk , and $M \in \text{MS}_{\text{PDPKS}}$, and outputs a signature Σ .

PDPKS.Verify: The verification algorithm takes pp , dpk , M , and Σ as input, and outputs 1 (valid) or 0 (invalid).

Correctness. It requires that for any $\lambda \in \mathbb{N}$, any $\text{pp} \leftarrow \text{PDPKS.Setup}(1^\lambda)$, any $(\text{mpk}, \text{msk}) \leftarrow \text{MasterKeyGen}(\text{pp})$, any $\text{dpk} \leftarrow \text{DpkDerive}(\text{pp}, \text{mpk})$, and any $M \in \text{MS}_{\text{PDPKS}}$,

$$\text{DpkCheck}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk}) = 1 \text{ and } \text{PDPKS.Verify}(\text{pp}, \text{dpk}, M, \Sigma) = 1$$

hold with overwhelming probability in the security parameter λ , where $\Sigma \leftarrow \text{PDPKS.Sign}(\text{pp}, \text{dpk}, \text{dsk}, M)$ and $\text{dsk} \leftarrow \text{DskDerive}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk})$.

Next, we define unforgeability as follows. We explicitly return $\perp$ if the DpkCheck algorithm returns 0 for derived secret key corruption queries. Note that $\text{DpkCheck}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk}) = 1$

holds for $\text{dpk} \in L_{\text{dpk}}$. Thus, the challenger does not return $\perp$ for signing queries. Since it is required that $\mathcal{A}$ did not send (dpk^*, M^*) as a signing query, the following does not capture strongly unforgeability.

Definition 1 (Unforgeability). *Let $\mathcal{A}$ be an adversary and $\mathcal{C}$ be the challenger. $\mathcal{C}$ runs $\text{pp} \leftarrow \text{PDPKS.Setup}(1^\lambda)$ and $(\text{mpk}, \text{msk}) \leftarrow \text{MasterKeyGen}(\text{pp})$, and gives (pp, mpk) to $\mathcal{A}$. $\mathcal{C}$ initializes $L_{\text{dpk}} := \emptyset$. $\mathcal{A}$ is allowed to issue the following queries.*

Derived Public Key Check Query: $\mathcal{A}$ sends dpk to $\mathcal{C}$. $\mathcal{C}$ returns the result of $\text{DpkCheck}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk})$. If $\text{DpkCheck}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk}) = 1$, then $\mathcal{C}$ updates $L_{\text{dpk}} \leftarrow L_{\text{dpk}} \cup \{\text{dpk}\}$.

Derived Secret Key Corruption Query: $\mathcal{A}$ sends $\text{dpk} \in L_{\text{dpk}}$ to $\mathcal{C}$. $\mathcal{C}$ returns $\perp$ if $\text{DpkCheck}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk}) = 0$. Otherwise, $\mathcal{C}$ returns $\text{dsk} \leftarrow \text{DskDerive}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk})$.

Signing Query: $\mathcal{A}$ sends $M \in \text{MS}_{\text{PDPKS}}$ and $\text{dpk} \in L_{\text{dpk}}$ to $\mathcal{C}$. $\mathcal{C}$ returns $\Sigma \leftarrow \text{PDPKS.Sign}(\text{pp}, \text{dpk}, \text{dsk}, M)$ where $\text{dsk} \leftarrow \text{DskDerive}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk})$.

Finally, $\mathcal{A}$ outputs $(\text{dpk}^*, M^*, \Sigma^*)$ where $M^* \in \text{MS}_{\text{PDPKS}}$. $\mathcal{A}$ wins if $\text{dpk}^* \in L_{\text{dpk}}$, $\text{PDPKS.Verify}(\text{pp}, \text{dpk}^*, M^*, \Sigma^*) = 1$, $\mathcal{A}$ did not send dpk^* as a derived secret key corruption query, and $\mathcal{A}$ did not send (dpk^*, M^*) as a signing query. The advantage is defined as

$$\text{Adv}_{\mathcal{A}, \text{PDPKS}}^{\text{unforge}}(\lambda) = \Pr[\mathcal{A} \text{ wins}]$$

We say that a PDPKS scheme PDPKS is unforgeable if $\text{Adv}_{\mathcal{A}, \text{PDPKS}}^{\text{unforge}}(\lambda)$ is negligible in the security parameter λ for all PPT adversaries $\mathcal{A}$.

Next, we define unlinkability as follows. As in unforgeability, we explicitly return $\perp$ if the DpkCheck algorithm returns 0 for derived secret key corruption queries.

Definition 2 (Unlinkability). *Let $\mathcal{A}$ be an adversary and $\mathcal{C}$ be the challenger. $\mathcal{C}$ runs $\text{pp} \leftarrow \text{PDPKS.Setup}(1^\lambda)$, $(\text{mpk}_0, \text{msk}_0) \leftarrow \text{MasterKeyGen}(\text{pp})$, and $(\text{mpk}_1, \text{msk}_1) \leftarrow \text{MasterKeyGen}(\text{pp})$, $\mathcal{C}$ chooses $b \leftarrow \{0, 1\}$ and computes $\text{dpk}^* \leftarrow \text{DpkDerive}(\text{pp}, \text{mpk}_b)$. $\mathcal{C}$ initializes $L_{\text{dpk},0} := \emptyset$ and $L_{\text{dpk},1} := \emptyset$. We remark that $\text{dpk}^* \notin L_{\text{dpk},0} \cup L_{\text{dpk},1}$. $\mathcal{C}$ gives $(\text{pp}, \text{mpk}_0, \text{mpk}_1, \text{dpk}^*)$ to $\mathcal{A}$. $\mathcal{A}$ is allowed to issue the following queries.*

Derived Public Key Check Query: $\mathcal{A}$ sends $\text{dpk} \neq \text{dpk}^*$ and index $i \in \{0, 1\}$ to $\mathcal{C}$. $\mathcal{C}$ returns the result of $\text{DpkCheck}(\text{pp}, \text{mpk}_i, \text{msk}_i, \text{dpk})$. If $\text{DpkCheck}(\text{pp}, \text{mpk}_i, \text{msk}_i, \text{dpk}) = 1$, then $\mathcal{C}$ updates $L_{\text{dpk},i} \leftarrow L_{\text{dpk},i} \cup \{\text{dpk}\}$.

Derived Secret Key Corruption Query: $\mathcal{A}$ sends $\text{dpk} \in L_{\text{dpk},0} \cup L_{\text{dpk},1}$ to $\mathcal{C}$. $\mathcal{C}$ returns $\perp$ if $\text{DpkCheck}(\text{pp}, \text{mpk}_i, \text{msk}_i, \text{dpk}) = 0$ where $\text{dpk} \in L_{\text{dpk},i}$. Otherwise, $\mathcal{C}$ returns $\text{dsk} \leftarrow \text{DskDerive}(\text{pp}, \text{mpk}_i, \text{msk}_i, \text{dpk})$.

Signing Query: $\mathcal{A}$ sends $M \in \text{MS}_{\text{PDPKS}}$ and $\text{dpk} \in L_{\text{dpk},0} \cup L_{\text{dpk},1} \cup \{\text{dpk}^*\}$ to $\mathcal{C}$. $\mathcal{C}$ returns $\Sigma \leftarrow \text{PDPKS.Sign}(\text{pp}, \text{dpk}, \text{dsk}, M)$. Here $\text{dsk} \leftarrow \text{DskDerive}(\text{pp}, \text{mpk}_i, \text{msk}_i, \text{dpk})$ where $\text{dpk} \in L_{\text{dpk},i}$ and $i = b$ if $\text{dpk} = \text{dpk}^*$.

Finally, $\mathcal{A}$ outputs $b' \in \{0, 1\}$. $\mathcal{A}$ wins if $b = b'$. The advantage is defined as

$$\text{Adv}_{\mathcal{A}, \text{PDPKS}}^{\text{unlink}}(\lambda) = |\Pr[b = b'] - 1/2|$$

We say that a PDPKS scheme PDPKS is unlinkable if $\text{Adv}_{\mathcal{A}, \text{PDPKS}}^{\text{unlink}}(\lambda)$ is negligible in the security parameter λ for all PPT adversaries $\mathcal{A}$.

3 New Definitions: Consistency and Outsider Strong Unforgeability

3.1 Definition of Consistency

To capture the condition that $\text{DpkCheck}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk}) = 0$ if dpk is not derived from mpk , we define consistency as follows. Here, $\mathcal{A}$ is not allowed to issue a derived public key check query unlike the other definition. This is reasonable because the DpkCheck algorithm is internally run to respond the query, and consistency considers a security of the DpkCheck algorithm. On the other hand, we need to guarantee that no adversary can break consistency even if the adversary has observed a valid signature. Thus, the adversary is allowed to issue signing queries but dpk is derived by the challenger.

Definition 3 (Consistency). *Let $\mathcal{A}$ be an adversary and $\mathcal{C}$ be the challenger. $\mathcal{C}$ runs $\text{pp} \leftarrow \text{PDPKS.Setup}(1^\lambda)$, $(\text{mpk}_0, \text{msk}_0) \leftarrow \text{MasterKeyGen}(\text{pp})$, and $(\text{mpk}_1, \text{msk}_1) \leftarrow \text{MasterKeyGen}(\text{pp})$, and gives $(\text{pp}, \text{mpk}_0, \text{mpk}_1)$ to $\mathcal{A}$. $\mathcal{A}$ is allowed to issue the following queries.*

Signing Query: $\mathcal{A}$ sends $M \in \text{MS}_{\text{PDPKS}}$ and index $i \in \{0, 1\}$ to $\mathcal{C}$. $\mathcal{C}$ runs $\text{dpk} \leftarrow \text{DpkDerive}(\text{pp}, \text{mpk}_i)$, $\text{dsk} \leftarrow \text{DskDerive}(\text{pp}, \text{mpk}_i, \text{msk}_i, \text{dpk})$, and $\Sigma \leftarrow \text{PDPKS.Sign}(\text{pp}, \text{dpk}, \text{dsk}, M)$, and returns (dpk, Σ) to $\mathcal{A}$.

Finally, $\mathcal{A}$ outputs dpk^* . $\mathcal{A}$ wins if $\text{DpkCheck}(\text{pp}, \text{mpk}_0, \text{msk}_0, \text{dpk}^*) = 1$ and $\text{DpkCheck}(\text{pp}, \text{mpk}_1, \text{msk}_1, \text{dpk}^*) = 1$ hold. The advantage is defined as

$$\text{Adv}_{\mathcal{A}, \text{PDPKS}}^{\text{consist}}(\lambda) = \Pr[\mathcal{A} \text{ wins}]$$

We say that a PDPKS scheme PDPKS is consistent if $\text{Adv}_{\mathcal{A}, \text{PDPKS}}^{\text{consist}}(\lambda)$ is negligible in the security parameter λ for all PPT adversaries $\mathcal{A}$.

Analysis of Previous PDPKS schemes: Here, we demonstrate whether previous PDPKS schemes provide consistency or not. We give a brief analysis and decline to give a formal proof here. The pairing-based Liu et al. PDPKS scheme [37] is briefly described as follows (Appendix may help the reader to understand the scheme). Let $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ be a pairing where $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ be groups with prime order p , and $g_1 \in \mathbb{G}_1$ and $g_2 \in \mathbb{G}_2$ be generators. $\text{mpk} = (\text{mpk}_1, \text{mpk}_2) = (g_2^\alpha, g_2^\beta) \in \mathbb{G}_2^2$ and $\text{msk} = (\text{msk}_1, \text{msk}_2) = (\alpha, \beta) \in \mathbb{Z}_p^2$. To derive dpk , choose $r \xleftarrow{\$} \mathbb{Z}_p$ and compute $\text{dpk} = (\text{dpk}_1, \text{dpk}_2) = (g_2^r, \text{mpk}_2 \cdot g_2^{\text{Hash}(g_2^r, \text{mpk}_1^r)})$. Here, Hash is modeled as a random oracle. $\text{mpk}_1^r = g_2^{\alpha r} = \text{dpk}_1^{\text{msk}_1}$ can be seen as a non-interactive key exchange (NIKE) [26] key and $\text{mpk}_2 \cdot g_2^{\text{Hash}(g_2^r, \text{mpk}_1^r)}$ can be seen as a Pedersen commitment for mpk_2 with the randomness (decommit) $\text{Hash}(g_2^r, \text{mpk}_1^r)$.

To check whether dpk is derived from mpk , check whether $\text{dpk}_2 = \text{mpk}_2 \cdot g_2^{\text{Hash}(\text{dpk}_1, \text{dpk}_1^{\text{msk}_1})}$ holds or not. Here, $\text{dpk}_1^{\text{msk}_1} = g_2^{r\alpha}$ is the NIKE key. If dpk is linked to two different master public keys $\text{mpk}_0 = (g_2^{\alpha_0}, g_2^{\beta_0})$ and $\text{mpk}_1 = (g_2^{\alpha_1}, g_2^{\beta_1})$, then $\text{dpk}_2 = g_2^{\beta_0} \cdot g_2^{\text{Hash}(\text{dpk}_1, \text{dpk}_1^{\alpha_0})} = g_2^{\beta_1} \cdot g_2^{\text{Hash}(\text{dpk}_1, \text{dpk}_1^{\alpha_1})}$ holds. Since Hash is modeled as a random oracle, $h_0 = \text{Hash}(\text{dpk}_1, \text{dpk}_1^{\alpha_0})$ and $h_1 = \text{Hash}(\text{dpk}_1, \text{dpk}_1^{\alpha_1})$ are uniformly distributed over $\mathbb{Z}_p$ and $h_0 \neq h_1$. Thus, the probability that $\text{dpk}_2 = g_2^{\beta_0 + h_0} = g_2^{\beta_1 + h_1}$ holds is negligible. This implies the Liu et al. scheme is consistent. The Liu et al. lattice-based PDPKS scheme [35] also provides the consistency if the underlying hash functions are modeled as random oracles. Briefly, dpk is computed by a hash value of a plaintext t and the ciphertext τ of the underlying PKE scheme, and τ is contained in dpk . That is, $\text{Hash}(t, \tau)$ can be (informally) seen as a shared key because a master secret key (decryption key of the PKE scheme) holder can

obtain t from τ . As in the pairing-based scheme, the probability that a random value (generated via the random oracle) coincidentally satisfies a checking equation is negligible. To sum up, previous PDPKS schemes are consistent if the underlying hash function is modeled as a random oracle.

3.2 Definition of Outsider Strong Unforgeability

Next, we define outsider strong unforgeability as follows. As mentioned before, we weaken unforgeability in the sense that $\mathcal{C}$ sends dpk^* to $\mathcal{A}$, and strengthen unforgeability in the sense that $\mathcal{A}$ is allowed to issue a signing query (dpk^*, M^*) .

Definition 4 (Outsider Strong Unforgeability). *Let $\mathcal{A}$ be an adversary and $\mathcal{C}$ be the challenger. $\mathcal{C}$ runs $\text{pp} \leftarrow \text{PDPKS.Setup}(1^\lambda)$ and $(\text{mpk}, \text{msk}) \leftarrow \text{MasterKeyGen}(\text{pp})$, computes $\text{dpk}^* \leftarrow \text{DpkDerive}(\text{pp}, \text{mpk})$, and gives $(\text{pp}, \text{mpk}, \text{dpk}^*)$ to $\mathcal{A}$. $\mathcal{C}$ initializes $L_{\text{dpk}} := \{\text{dpk}^*\}$ and $L_{\text{Sig}} := \emptyset$. $\mathcal{A}$ is allowed to issue the following queries.*

Derived Public Key Check Query: $\mathcal{A}$ sends dpk to $\mathcal{C}$ ($\text{dpk} = \text{dpk}^*$ is allowed). $\mathcal{C}$ returns the result of $\text{DpkCheck}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk})$. If $\text{DpkCheck}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk}) = 1$, then $\mathcal{C}$ updates $L_{\text{dpk}} \leftarrow L_{\text{dpk}} \cup \{\text{dpk}\}$.

Derived Secret Key Corruption Query: $\mathcal{A}$ sends $\text{dpk} \in L_{\text{dpk}} \setminus \{\text{dpk}^*\}$ to $\mathcal{C}$. $\mathcal{C}$ returns $\perp$ if $\text{DpkCheck}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk}) = 0$. Otherwise, $\mathcal{C}$ returns $\text{dsk} \leftarrow \text{DskDerive}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk})$.

Signing Query: $\mathcal{A}$ sends $M \in \text{MS}_{\text{PDPKS}}$ and $\text{dpk} \in L_{\text{dpk}}$ to $\mathcal{C}$. $\mathcal{C}$ returns $\Sigma \leftarrow \text{PDPKS.Sign}(\text{pp}, \text{dpk}, \text{dsk}, M)$ where $\text{dsk} \leftarrow \text{DskDerive}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk})$. Moreover, if $\text{dpk} = \text{dpk}^*$, $\mathcal{C}$ updates $L_{\text{Sig}} := L_{\text{Sig}} \cup (M, \Sigma)$.

Finally, $\mathcal{A}$ outputs (M^*, Σ^*) where $M^* \in \text{MS}_{\text{PDPKS}}$. $\mathcal{A}$ wins if $\text{PDPKS.Verify}(\text{pp}, \text{dpk}^*, M^*, \Sigma^*) = 1$ and $(M^*, \Sigma^*) \notin L_{\text{Sig}}$. The advantage is defined as

$$\text{Adv}_{\mathcal{A}, \text{PDPKS}}^{\text{outsider-strong-unforge}}(\lambda) = \Pr[\mathcal{A} \text{ wins}]$$

We say that a PDPKS scheme PDPKS is strongly unforgeable for outsider if $\text{Adv}_{\mathcal{A}, \text{PDPKS}}^{\text{outsider-strong-unforge}}(\lambda)$ is negligible in the security parameter λ for all PPT adversaries $\mathcal{A}$.

4 Proposed Generic Construction

In this section, we give the proposed generic construction. Let $\text{Sig} = (\text{Sig.Setup}, \text{Sig.KeyGen}, \text{Sig.Sign}, \text{Sig.Verify})$ and $\text{PKE} = (\text{PKE.Setup}, \text{PKE.KeyGen}, \text{PKE.Enc}, \text{PKE.Dec})$ be a signature scheme and a PKE scheme, respectively. A payer (who generates dpk) chooses a verification key and signing key pair (vk, sigk) and encrypts sigk by using mpk where $\text{mpk} = \text{PKE.pk}$. One may think that the construction is trivial since a signing key is sent via a secure channel encrypted by PKE.pk . This intuition is true and it well explains the fact that the design is conceptually much simpler than known PDPKS constructions. However, the security proof is not trivial, e.g., the underlying signature scheme is required to provide the S-CEO security. First, we give an intuition of the proposed generic construction as follows.

- The DpkDerive algorithm internally generates a verification key and signing key pair (vk, sigk) on the fly. Let $\text{mpk} = \text{PKE.pk}$ and $\text{msk} = \text{PKE.dk}$. The algorithm encrypts sigk by using $\text{mpk} = \text{PKE.pk}$ such that $C_{\text{PKE}} \leftarrow \text{PKE.Enc}(\text{PKE.pk}, \text{sigk})$. Set $\text{dpk} = (\text{vk}, C_{\text{PKE}})$. Now, vk is independent of mpk but C_{PKE} depends on mpk . To hide information of mpk , we assume

that the underlying PKE scheme is key private. Moreover, the underlying signature scheme is required to provide the S-CEO security (See Section 5 for details).

- The DskDerive algorithm decrypts C_{PKE} by using $\text{msk} = \text{PKE.dk}$ such that $\text{sigk} \leftarrow \text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})$. The algorithm also checks whether sigk is a valid signing key for the verification key vk by generating a signature on a random message. To provide the consistency, we assume that the underlying PKE scheme is robust. Then, $\perp \leftarrow \text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})$ holds with overwhelming probability if $\text{PKE.dk} \neq \text{msk}$.
- To sign a message (transaction) M , the PDPKS.Sign algorithm simply signs M using sigk . Then, mpk is not required for verifying a PDPKS signature. To provide strong unforgeability, we assume that the underlying signature scheme is strongly unforgeable. Moreover, the underlying signature scheme is required to provide the S-CEO security (See Section 5 for details).

Here, we give the proposed generic construction.

PDPKS.Setup(1^λ): Run $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda)$ and $\text{pp}_{\text{PKE}} \leftarrow \text{PKE.Setup}(1^\lambda)$, and output $\text{pp} = (\text{pp}_{\text{Sig}}, \text{pp}_{\text{PKE}})$.

MasterKeyGen(pp): Parse $\text{pp} = (\text{pp}_{\text{Sig}}, \text{pp}_{\text{PKE}})$. Run $(\text{PKE.pk}, \text{PKE.dk}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$ and output $(\text{mpk}, \text{msk}) = (\text{PKE.pk}, \text{PKE.dk})$.

DpkDerive(pp, mpk): Parse $\text{pp} = (\text{pp}_{\text{Sig}}, \text{pp}_{\text{PKE}})$ and $\text{mpk} = \text{PKE.pk}$. Run $(\text{vk}, \text{sigk}) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$ and $C_{\text{PKE}} \leftarrow \text{PKE.Enc}(\text{PKE.pk}, \text{sigk})$. Output $\text{dpk} = (\text{vk}, C_{\text{PKE}})$.

DpkCheck(pp, mpk, msk, dpk): Parse $\text{pp} = (\text{pp}_{\text{Sig}}, \text{pp}_{\text{PKE}})$, $\text{mpk} = \text{PKE.pk}$, $\text{msk} = \text{PKE.dk}$, and $\text{dpk} = (\text{vk}, C_{\text{PKE}})$. Output 0 if $\perp \leftarrow \text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})$. Otherwise, let $\text{sigk} \leftarrow \text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})$. Choose $m \xleftarrow{\$} \text{MS}_{\text{Sig}}$,¹³ and output the result of $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}, m))$.

DskDerive(pp, mpk, msk, dpk): Parse $\text{pp} = (\text{pp}_{\text{Sig}}, \text{pp}_{\text{PKE}})$, $\text{mpk} = \text{PKE.pk}$, $\text{msk} = \text{PKE.dk}$, and $\text{dpk} = (\text{vk}, C_{\text{PKE}})$. Output $\text{dsk} = \perp$ if $\text{DpkCheck}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk}) = 0$. Otherwise, if $\text{DpkCheck}(\text{pp}, \text{mpk}, \text{msk}, \text{dpk}) = 1$, then sigk are obtained. Output $\text{dsk} = \text{sigk}$.

PDPKS.Sign(pp, dpk, M , dsk): Parse $\text{pp} = (\text{pp}_{\text{Sig}}, \text{pp}_{\text{PKE}})$, $\text{dpk} = (\text{vk}, C_{\text{PKE}})$, and $\text{dsk} = \text{sigk}$. Output $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}, M)$.

PDPKS.Verify(pp, dpk, M , Σ): Parse $\text{pp} = (\text{pp}_{\text{Sig}}, \text{pp}_{\text{PKE}})$ and $\text{dpk} = (\text{vk}, C_{\text{PKE}})$. Output 1 if $\text{Sig.Verify}(\text{vk}, M, \Sigma) = 1$ holds, and 0 otherwise.

Before proving the security of the proposed generic construction, we clarify that the generic construction does not deviate the requirement given in [37] as follows.

Low-maintenance wallets with easy backup and recovery. Liu et al. mentioned that “*To backup his deterministic wallet, a user only needs to backup the (long-term) secret key (α, β) . When needed, he could reconstruct the complete wallet, by using α to scan the ledger to find which transaction-outputs are his coins, and using (α, β) to generate the corresponding derived signing keys.*”. Here, “scan” indicates some cryptographic operations including a

¹³In our construction, DpkCheck is a probabilistic algorithm though it is a deterministic algorithm in the original definition. This difference does not affect the security.

scalar multiplication αQ_r in their notation. Similarly, in the generic construction, a user only needs to backup the (long-term) secret key $\text{msk} = \text{PKE.dk}$. Here, “scan” indicates $\text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})$ is $\perp$ or not, and it can recover $\text{dsk} = \text{sigk}$.

Freshly generated cold addresses. Liu et al. mentioned that “*With the PDPKS scheme, a user can publish his (long-term) public key on a hot storage, without affecting the security or privacy. To use the cold address mechanism, he can easily generate derived verification keys as he needs. In addition, a user even can ask the payer to generate the derived verification keys for the transaction sending coins to him, and he only checks and ensures that no derived verification key is used more than once, by rejecting the transactions using repeated derived verification keys.*”. Since $(\text{vk}, \text{sigk}) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$ is chosen by the payer and hence is independent of the payee’s mpk , one may think that the payer can reuse the same pair across multiple payees. Here, such the attempt can be detected since $\text{dpk} = (\text{vk}, C_{\text{PKE}})$ is public. Moreover, the freshness in the Liu et al.’s PDPKS scheme relies on the assumption that the payer randomly chooses $r \xleftarrow{\$} \mathbb{Z}_p$ and distinct r is used for each key derivation. In this perspective, the freshness of our PDPKS construction relies on the same assumption that the payer uses a random value for generating $(\text{vk}, \text{sigk}) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$ and distinct randomness is used for each key derivation.

Trustless audits. Liu et al. mentioned that “*For a user with (long-term) secret key α , β revealing α to an auditor will enable the auditor to view all the transactions related to (long-term) public key. Note that revealing α just conceals the privacy-protection, without affecting the security at all.*”. In the generic construction, revealing $\text{msk} = \text{PKE.dk}$ affects the security since it can decrypt C_{PKE} and reveals $\text{dsk} = \text{sigk}$. However, the following approach implements trustless audits. The point is to prove that C_{PKE} is computed by using $\text{mpk} = \text{PKE.pk}$ for $\text{dpk} = (\text{vk}, C_{\text{PKE}})$. One easy way to connect transactions is to employ a zero-knowledge proof for the relation $\{((C_{\text{PKE}}, \text{PKE.pk}), (r_{\text{PKE}}, \text{sigk})) : (\text{PKE.pk}, \text{PKE.dk}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}}; r_{\text{PKE}}) \wedge \text{sigk} \leftarrow \text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})\}$, where $(C_{\text{PKE}}, \text{PKE.pk})$ is a (public) statement and $(r_{\text{PKE}}, \text{sigk})$ is the corresponding witness. The randomness r_{PKE} is used for running the PKE.KeyGen algorithm.¹⁴ Then, the validity of a transaction M can be checked by $\text{Sig.Verify}(\text{vk}, M, \Sigma)$ and the auditor can view all the transactions related to (long-term) public key mpk without affecting the security.¹⁵

5 Security Analysis

Obviously, correctness directly holds if Sig and PKE are correct. Next, we prove that the proposed construction is consistent.

Theorem 1. *The proposed construction is consistent if the underlying PKE scheme is strongly robust.*

Proof. Let $\mathcal{A}$ be an adversary of consistency and $\mathcal{C}$ be the challenger of strong robustness. We construct an algorithm $\mathcal{B}$ that breaks strong robustness by using $\mathcal{A}$ as follows. First, $\mathcal{C}$ generates $\text{pp}_{\text{PKE}} \leftarrow \text{PKE.Setup}(1^\lambda)$, $(\text{PKE.pk}_0, \text{PKE.dk}_0) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, and $(\text{PKE.pk}_1, \text{PKE.dk}_1)$

¹⁴We assume that msk contains the randomness r_{PKE} if the audit is necessary.

¹⁵Here, we assume that a proof is generated for a valid dpk . If the auditor needs to verify the validity of dpk , then the relation needs to be modified: $\{((\text{vk}, C_{\text{PKE}}, \text{PKE.pk}, m, \sigma), (r_{\text{PKE}}, r_{\text{Sig}}, \text{sigk})) : (\text{PKE.pk}, \text{PKE.dk}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}}; r_{\text{PKE}}) \wedge \text{sigk} \leftarrow \text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}}) \wedge \sigma \leftarrow \text{Sig.Sign}(\text{sigk}, m; r_{\text{Sig}})\}$, where m is a random message. The auditor additionally checks $\text{Sig.Verify}(\text{vk}, m, \sigma) = 1$.

$\leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, and gives $(\text{pp}_{\text{PKE}}, \text{PKE.pk}_0, \text{PKE.pk}_1)$ to $\mathcal{B}$. $\mathcal{B}$ sets $\text{mpk}_0 = \text{PKE.pk}_0$ and $\text{mpk}_1 = \text{PKE.pk}_1$. $\mathcal{B}$ runs $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda)$ and sets $\text{pp} = (\text{pp}_{\text{Sig}}, \text{pp}_{\text{PKE}})$. $\mathcal{B}$ gives $(\text{pp}, \text{mpk}_0, \text{mpk}_1)$ to $\mathcal{A}$. $\mathcal{B}$ answers queries issued by $\mathcal{A}$ as follows.

Signing Query: $\mathcal{A}$ sends $M \in \text{MS}_{\text{PDKS}}$ and index $i \in \{0, 1\}$ to $\mathcal{B}$. $\mathcal{B}$ runs $(\text{vk}, \text{sigk}) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$, $C_{\text{PKE}} \leftarrow \text{PKE.Enc}(\text{PKE.pk}_i, \text{sigk})$, and $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}, M)$. $\mathcal{B}$ returns $\text{dpk} = (\text{vk}, C_{\text{PKE}})$ and Σ to $\mathcal{A}$.

Finally, $\mathcal{A}$ outputs $\text{dpk}^* = (\text{vk}^*, C_{\text{PKE}}^*)$. Since $\text{DpkCheck}(\text{pp}, \text{mpk}_0, \text{msk}_0, \text{dpk}^*) = 1$ and $\text{DpkCheck}(\text{pp}, \text{mpk}_1, \text{msk}_1, \text{dpk}^*) = 1$ hold, the decryption results of C_{PKE}^* by using PKE.dk_0 and PKE.dk_1 are both non- $\perp$. $\mathcal{B}$ outputs C_{PKE}^* and breaks strong robustness. $\square$

Theorem 2. *The proposed construction is strongly unforgeable for outsider if the underlying PKE scheme is CCA secure and the underlying signature scheme is S-CEO secure and strongly unforgeable.*

Before giving our security proof, we give an intuition of the proof. Let $\text{dpk}^* = (\text{vk}^*, C_{\text{PKE}}^*)$ where C_{PKE}^* is a ciphertext of sigk^* . Our final goal is to reduce strong unforgeability. Then, the challenger of the signature scheme sends the challenge verification key vk^* to the reduction algorithm. Since C_{PKE}^* is a ciphertext of sigk^* , the reduction algorithm cannot produce dpk^* . Thus, before reducing to strong unforgeability, we replace C_{PKE}^* to a ciphertext of $0^{|\text{sigk}^*|}$ due to the IND-CCA security of the PKE scheme. Here, we implicitly assume that for any $(\text{vk}, \text{sigk}) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$, $|\text{sigk}|$ is the same.¹⁶ Now, the reduction algorithm can produce dpk^* by obtaining vk^* from the challenger of the signature scheme and by computing $C_{\text{PKE}}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, 0^{|\text{sigk}^*|})$. The remaining issue is how to respond a derived public key check query and a derived secret key corruption query for $\text{dpk} = (\text{vk}, C_{\text{PKE}}^*)$ where $\text{vk} \neq \text{vk}^*$ but vk is a valid verification key relative to sigk^* . Since the reduction algorithm needs to return 1 for the derived public key check query dpk , the reduction algorithm needs to return sigk^* for the derived secret key corruption query dpk . Thus, we need to guarantee that $\mathcal{A}$ does not produce such a dpk . Now, it is the turn of the S-CEO security. If an adversary produces such a dpk , the reduction algorithm outputs vk that breaks the S-CEO security. Finally, we show that an algorithm exists that breaks strong unforgeability of the signature scheme.

Proof. We use a game sequence Game_0 , Game_1 , and Game_2 . Let E_i be an event that $\mathcal{A}$ wins in Game_i .

Game₀. This is the security game of outsider strong unforgeability. By definition,

$$\text{Adv}_{\mathcal{A}, \text{PDKS}}^{\text{outsider-strong-unforge}}(\lambda) = \Pr[E_0]$$

Game₁. This is the same as Game_0 except that C_{PKE}^* is replaced by a ciphertext of $0^{|\text{sigk}^*|}$. We show that there exists an algorithm $\mathcal{B}_1$ such that $|\Pr[E_0] - \Pr[E_1]| \leq \text{Adv}_{\mathcal{B}_1, \text{PKE}}^{\text{IND-CCA}}(\lambda)$ as follows. Let $\mathcal{A}$ be an adversary of the outsider strong unforgeability and $\mathcal{C}$ be the challenger of the PKE scheme. $\mathcal{C}$ generates $\text{pp}_{\text{PKE}} \leftarrow \text{PKE.Setup}(1^\lambda)$ and $(\text{PKE.pk}, \text{PKE.dk}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, and gives $(\text{pp}_{\text{PKE}}, \text{PKE.pk})$ to $\mathcal{B}_1$. $\mathcal{B}_1$ runs $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda)$ and sets $\text{pp} = (\text{pp}_{\text{Sig}}, \text{pp}_{\text{PKE}})$ and $\text{mpk} = \text{PKE.pk}$. $\mathcal{B}_1$ runs $(\text{vk}^*, \text{sigk}^*) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$ and sends $(M_0^*, M_1^*) = (\text{sigk}^*, 0^{|\text{sigk}^*|})$ to $\mathcal{C}$ as the challenge query. $\mathcal{C}$ chooses $b \xleftarrow{\$} \{0, 1\}$, computes the challenge ciphertext $C_{\text{PKE}}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, M_b^*)$, and returns C_{PKE}^* to $\mathcal{B}_1$. $\mathcal{B}_1$ sets $\text{dpk}^* = (\text{vk}^*, C_{\text{PKE}}^*)$ and gives $(\text{pp}, \text{mpk}, \text{dpk}^*)$ to $\mathcal{A}$. $\mathcal{B}_1$ initializes $L_{\text{dpk}} = \{\text{dpk}^*\}$. $\mathcal{B}_1$ answers queries issued by $\mathcal{A}$ as follows.

¹⁶This is not a strong requirement. Even if each signing key has a different size, we can artificially add some paddings.

Derived Public Key Check Query: $\mathcal{A}$ sends $\text{dpk} = (\text{vk}, C_{\text{PKE}})$ to $\mathcal{B}_1$. If $\text{dpk} = \text{dpk}^*$, then $\mathcal{B}_1$ returns 1. Otherwise, if $\text{dpk} \neq \text{dpk}^*$ and $C_{\text{PKE}} = C_{\text{PKE}}^*$, then $\mathcal{B}_1$ chooses $m \xleftarrow{\$} \text{MS}_{\text{Sig}}$. If $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}^*, m)) = 1$, then $\mathcal{B}_1$ updates $L_{\text{dpk}} \leftarrow L_{\text{dpk}} \cup \{\text{dpk}\}$ and returns 1. Otherwise, $\mathcal{B}_1$ returns 0. If $\text{vk} \neq \text{vk}^*$ and $C_{\text{PKE}} \neq C_{\text{PKE}}^*$, $\mathcal{B}_1$ sends C_{PKE} to $\mathcal{C}$ as a decryption query. $\mathcal{C}$ returns $\text{sigk} \leftarrow \text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})$ to $\mathcal{B}_1$. $\mathcal{B}_1$ returns 0 if $\text{sigk} = \perp$. If $\text{sigk} \neq \perp$, $\mathcal{B}_1$ chooses $m \xleftarrow{\$} \text{MS}_{\text{Sig}}$. If $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}^*, m)) = 1$, then $\mathcal{B}_1$ updates $L_{\text{dpk}} \leftarrow L_{\text{dpk}} \cup \{\text{dpk}\}$ and returns 1. Otherwise, $\mathcal{B}_1$ returns 0.

Derived Secret Key Corruption Query: $\mathcal{A}$ sends $\text{dpk} = (\text{vk}, C_{\text{PKE}})$ where $\text{dpk} \in L_{\text{dpk}} \setminus \{\text{dpk}^*\}$. If $\text{dpk} \neq \text{dpk}^*$ and $C_{\text{PKE}} = C_{\text{PKE}}^*$, then $\mathcal{B}_1$ chooses $m \xleftarrow{\$} \text{MS}_{\text{Sig}}$. If $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}^*, m)) = 1$, then $\mathcal{B}_1$ returns sigk^* , and $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}^*, m)) = 0$, then $\mathcal{B}_1$ returns $\perp$. If $\text{dpk} \neq \text{dpk}^*$ and $C_{\text{PKE}} \neq C_{\text{PKE}}^*$, then $\mathcal{B}_1$ sends C_{PKE} to $\mathcal{C}$ as a decryption query. $\mathcal{C}$ returns $\text{sigk} \leftarrow \text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})$ to $\mathcal{B}_1$. $\mathcal{B}_1$ returns $\perp$ if $\text{sigk} = \perp$. Otherwise, $\mathcal{B}_1$ chooses $m \xleftarrow{\$} \text{MS}_{\text{Sig}}$ and returns sigk to $\mathcal{A}$ if $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}, m)) = 1$, and $\perp$ if $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}, m)) = 0$.

Signing Query: $\mathcal{A}$ sends $M \in \text{MS}_{\text{PDPKS}}$ and $\text{dpk} = (\text{vk}, C_{\text{PKE}}) \in L_{\text{dpk}}$ to $\mathcal{B}_1$. Since $\text{dpk} \in L_{\text{dpk}}$, $\mathcal{A}$ has sent dpk as a derived public key check query, and $\mathcal{B}_1$ retrieves $\text{dsk} = \text{sigk}$ by internally issuing a derived secret key corruption query or uses sigk if $\mathcal{A}$ has issued dpk as a derived secret key corruption query. $\mathcal{B}_1$ returns $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}, M)$.

If $b = 0$, then $\mathcal{B}_1$ simulates Game_0 and if $b = 1$, $\mathcal{B}_1$ simulates Game_1 . Thus, $|\Pr[E_0] - \Pr[E_1]| \leq \text{Adv}_{\mathcal{B}_1, \text{PKE}}^{\text{IND-CCA}}(\lambda)$ holds.

Game₂. This is the same as Game_1 except that the response of derived public key check queries is changed. Let $\text{dpk} = (\text{vk}, C_{\text{PKE}})$ be a derived public key check query. If $\text{vk} \neq \text{vk}^*$, $C_{\text{PKE}} = C_{\text{PKE}}^*$, and $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}^*, m)) = 1$ holds for a random choice of $m \in \text{MS}_{\text{Sig}}$, then the challenger output 0. If this event does not happen, Game_1 and Game_2 are identical. Thus, $|\Pr[E_1] - \Pr[E_2]| \leq \Pr[E]$ holds where $\Pr[E]$ is the probability that the event happens since $\Pr[E_2] = \Pr[E_2 | \neg E] \Pr[\neg E] + \Pr[E_2 | E] \Pr[E] \leq \Pr[E_2 | \neg E] + \Pr[E] = \Pr[E_1] + \Pr[E]$. We show that $\Pr[E]$ is negligible if Sig provides the S-CEO security as follows. Let $\mathcal{A}$ be an adversary of outsider strong unforgeability and $\mathcal{C}$ be the challenger of the signature scheme. We construct an algorithm $\mathcal{B}_2$ that breaks the S-CEO security if the event happens as follows. $\mathcal{C}$ runs $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda)$ and $(\text{vk}^*, \text{sigk}^*) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$ and sends $(\text{pp}_{\text{Sig}}, \text{vk}^*)$ to $\mathcal{B}_2$. $\mathcal{B}_2$ runs $\text{pp}_{\text{PKE}} \leftarrow \text{PKE.Setup}(1^\lambda)$ and $(\text{PKE.pk}, \text{PKE.dk}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, computes $C_{\text{PKE}}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, 0^{|\text{sigk}^*|})$, sets $\text{pp} = (\text{pp}_{\text{Sig}}, \text{pp}_{\text{PKE}})$, $\text{mpk} = \text{PKE.pk}$, and $\text{dpk}^* = (\text{vk}^*, C_{\text{PKE}}^*)$, and gives $(\text{pp}, \text{mpk}, \text{dpk}^*)$ to $\mathcal{A}$. $\mathcal{B}_2$ initializes $L_{\text{dpk}} = \{\text{dpk}^*\}$. $\mathcal{B}_2$ answers queries issued by $\mathcal{A}$ as follows.

Derived Public Key Check Query: $\mathcal{A}$ sends $\text{dpk} = (\text{vk}, C_{\text{PKE}})$ to $\mathcal{B}_2$. If $\text{dpk} = \text{dpk}^*$, then $\mathcal{B}_2$ returns 1. Otherwise, if $\text{dpk} \neq \text{dpk}^*$ and $C_{\text{PKE}} = C_{\text{PKE}}^*$, then $\mathcal{B}_2$ chooses $m \xleftarrow{\$} \text{MS}_{\text{Sig}}$ and sends m to $\mathcal{C}$ as a signing query. $\mathcal{C}$ returns $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}^*, m)$. If $\text{Sig.Verify}(\text{vk}, m, \Sigma) = 1$ (i.e., the event happens), then $\mathcal{B}_2$ outputs (vk, m, Σ) that breaks the S-CEO security. Otherwise, if $\text{Sig.Verify}(\text{vk}, m, \Sigma) = 0$, then $\mathcal{B}_2$ returns 0. If $\text{dpk} \neq \text{dpk}^*$ and $C_{\text{PKE}} \neq C_{\text{PKE}}^*$, then $\mathcal{B}_2$ computes $\text{sigk} \leftarrow \text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})$. $\mathcal{B}_2$ returns 0 if $\text{sigk} = \perp$. Otherwise, if $\text{sigk} \neq \perp$, $\mathcal{B}_2$ chooses $m \xleftarrow{\$} \text{MS}_{\text{Sig}}$. If $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}, m)) = 1$, then $\mathcal{B}_2$ updates $L_{\text{dpk}} \leftarrow L_{\text{dpk}} \cup \{\text{dpk}\}$ and returns 1. Otherwise, if $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}, m)) = 0$, then $\mathcal{B}_2$ returns 0.

Derived Secret Key Corruption Query: $\mathcal{A}$ sends $\text{dpk} = (\text{vk}, C_{\text{PKE}})$ where $\text{dpk} \in L_{\text{dpk}} \setminus$

$\{\text{dpk}^*\}$. If $\text{dpk} \neq \text{dpk}^*$ and $C_{\text{PKE}} = C_{\text{PKE}}^*$, then $\mathcal{B}_2$ has broken the S-CEO security. Otherwise, $\mathcal{B}_2$ computes $\text{sigk} \leftarrow \text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})$ and returns sigk to $\mathcal{A}$.

Signing Query: $\mathcal{A}$ sends $M \in \text{MS}_{\text{PDPKS}}$ and $\text{dpk} = (\text{vk}, C_{\text{PKE}}) \in L_{\text{dpk}}$ to $\mathcal{B}_2$. If $\text{dpk} = \text{dpk}^*$, then $\mathcal{B}_2$ sends M to $\mathcal{C}$ as a signing query, obtains Σ , and returns Σ . Otherwise, $\mathcal{B}_2$ retrieves $\text{dsk} = \text{sigk}$ by internally issuing a derived secret key corruption query or uses sigk if $\mathcal{A}$ has issued dpk as a derived secret key corruption query. $\mathcal{B}_2$ returns $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}, M)$.

If the event happens, then $\mathcal{B}_2$ breaks the S-CEO security. Thus, $\Pr[E] \leq \text{Adv}_{\mathcal{B}_2, \text{Sig}}^{\text{S-CEO}}(\lambda)$ holds.

Finally, in Game_2 , we construct an algorithm $\mathcal{B}_3$ that breaks strong unforgeability, i.e., $\Pr[E_2] \leq \text{Adv}_{\mathcal{B}_3, \text{Sig}}^{\text{strong}}(\lambda)$, as follows. In this game, $\mathcal{B}_3$ returns 0 for a derived public key check query dpk where $\text{dpk} \neq \text{dpk}^*$ and $C_{\text{PKE}} = C_{\text{PKE}}^*$ due to the modification of the previous game. Let $\mathcal{A}$ be an adversary of outsider strong unforgeability and $\mathcal{C}$ be the challenger of the signature scheme. $\mathcal{C}$ runs $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda)$ and $(\text{vk}^*, \text{sigk}^*) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$ and sends $(\text{pp}_{\text{Sig}}, \text{vk}^*)$ to $\mathcal{B}_3$. $\mathcal{B}_3$ runs $\text{pp}_{\text{PKE}} \leftarrow \text{PKE.Setup}(1^\lambda)$ and $(\text{PKE.pk}, \text{PKE.dk}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, computes $C_{\text{PKE}}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, 0^{|\text{sigk}|})$, sets $\text{pp} = (\text{pp}_{\text{Sig}}, \text{pp}_{\text{PKE}})$, $\text{mpk} = \text{PKE.pk}$, and $\text{dpk}^* = (\text{vk}^*, C_{\text{PKE}}^*)$, and gives $(\text{pp}, \text{mpk}, \text{dpk}^*)$ to $\mathcal{A}$. $\mathcal{B}_3$ initializes $L_{\text{dpk}} = \{\text{dpk}^*\}$. $\mathcal{B}_3$ answers queries issued by $\mathcal{A}$ as follows.

Derived Public Key Check Query: $\mathcal{A}$ sends $\text{dpk} = (\text{vk}, C_{\text{PKE}})$ to $\mathcal{B}_3$. If $\text{dpk} = \text{dpk}^*$, then $\mathcal{B}_3$ returns 1. Otherwise, if $\text{dpk} \neq \text{dpk}^*$ and $C_{\text{PKE}} = C_{\text{PKE}}^*$, then $\mathcal{B}_3$ returns 0. Otherwise, $\mathcal{B}_3$ computes $\text{sigk} \leftarrow \text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})$. $\mathcal{B}_3$ returns 0 if $\text{sigk} = \perp$. Otherwise, $\mathcal{B}_3$ chooses $m \xleftarrow{\$} \text{MS}_{\text{Sig}}$. If $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}, m)) = 1$, then $\mathcal{B}_3$ updates $L_{\text{dpk}} \leftarrow L_{\text{dpk}} \cup \{\text{dpk}\}$ and returns 1. Otherwise, $\mathcal{B}_3$ returns 0.

Derived Secret Key Corruption Query: $\mathcal{A}$ sends $\text{dpk} = (\text{vk}, C_{\text{PKE}})$ where $\text{dpk} \in L_{\text{dpk}} \setminus \{\text{dpk}^*\}$. If $\text{dpk} \neq \text{dpk}^*$ and $C_{\text{PKE}} = C_{\text{PKE}}^*$, then $\mathcal{B}_3$ returns $\perp$. Otherwise, $\mathcal{B}_3$ computes $\text{sigk} \leftarrow \text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})$ and returns sigk .

Signing Query: $\mathcal{A}$ sends $M \in \text{MS}_{\text{PDPKS}}$ and $\text{dpk} = (\text{vk}, C_{\text{PKE}}) \in L_{\text{dpk}}$ to $\mathcal{B}_3$. If $\text{dpk} = \text{dpk}^*$, then $\mathcal{B}_3$ sends M to $\mathcal{C}$ as a signing query. $\mathcal{C}$ returns $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}^*, M)$ to $\mathcal{B}_3$. $\mathcal{B}_3$ returns Σ to $\mathcal{A}$. Otherwise, $\mathcal{B}_3$ retrieves $\text{dsk} = \text{sigk}$ by internally issuing a derived secret key corruption query or uses sigk if $\mathcal{A}$ has issued dpk as a derived secret key corruption query. $\mathcal{B}_3$ returns $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}, M)$.

Finally, $\mathcal{A}$ outputs (M^*, Σ^*) . $\mathcal{B}_3$ outputs (M^*, Σ^*) as a forgery and breaks strong unforgeability.

Now, we have

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{PDPKS}}^{\text{outsider-strong-unforge}}(\lambda) &= \Pr[E_0] \\ &= \Pr[E_0] - \Pr[E_1] + \Pr[E_1] - \Pr[E_2] + \Pr[E_2] \\ &\leq |\Pr[E_0] - \Pr[E_1]| + |\Pr[E_1] - \Pr[E_2]| + \Pr[E_2] \\ &\leq |\Pr[E_0] - \Pr[E_1]| + \Pr[E] + \Pr[E_2] \\ &\leq \text{Adv}_{\mathcal{B}_1, \text{PKE}}^{\text{IND-CCA}}(\lambda) + \text{Adv}_{\mathcal{B}_2, \text{Sig}}^{\text{S-CEO}}(\lambda) + \text{Adv}_{\mathcal{B}_3, \text{Sig}}^{\text{strong}}(\lambda) \end{aligned}$$

This concludes the proof. $\square$

Theorem 3. *The proposed construction is unlinkable if the underlying signature scheme is S-CEO secure and the underlying PKE scheme is CCA secure and key private.*

Before giving our security proof, we give an intuition of the proof. Basically, for the challenge derived public key $\text{dpk}^* = (\text{vk}^*, C_{\text{PKE}}^*)$, no information of mpk is revealed if the PKE scheme is key private. However, if an adversary issues a derived public key check query $\text{dpk} = (\text{vk}, C_{\text{PKE}}^*)$ and $i \in \{0, 1\}$ where $\text{vk} \neq \text{vk}^*$, the reduction algorithm needs to know not only whether vk is a valid verification key relative to sigk^* but also C_{PKE}^* is generated by mpk_i or not. This implies the reduction algorithm breaks key privacy without using the adversary, and the reduction algorithm fails the simulation. Thus, we need to change the game description where the reduction algorithm returns 0 for a derived public key check query $\text{dpk} = (\text{vk}, C_{\text{PKE}}^*)$ if $\text{vk} \neq \text{vk}^*$ regardless of i . Here, the reduction algorithm can respond 0 to the query regardless of i if vk is not a valid verification key relative to sigk^* . Here, if vk is a valid verification key relative to sigk^* , then we can construct an algorithm that breaks the S-CEO security, as in the security proof of outsider strong unforgeability. Thanks to the game modifications, the reduction algorithm (for key privacy) responds 0 for a derived public key check query $\text{dpk} = (\text{vk}, C_{\text{PKE}}^*)$ regardless of i .

Proof. We use a game sequence Game_0 , Game_1 , and Game_2 . Let E_i be an event that $\mathcal{A}$ wins in Game_i .

Game₀. This is the security game of unlinkability. By definition,

$$\text{Adv}_{\mathcal{A}, \text{PDPKS}}^{\text{unlink}}(\lambda) = |\Pr[E_0] - 1/2|$$

Game₁. This is the same as Game_0 except that C_{PKE}^* is replaced by a ciphertext of $0^{|\text{sigk}^*|}$. We show that there exists an algorithm $\mathcal{B}_1$ such that $|\Pr[E_0] - \Pr[E_1]| \leq \text{Adv}_{\mathcal{B}_1, \text{PKE}}^{\text{IND-CCA}}(\lambda)$ as follows. Let $\mathcal{A}$ be an adversary of unlinkability and $\mathcal{C}$ be the challenger of the PKE scheme. Note that the definition of key privacy contains CCA security but here $\mathcal{C}$ is the IND-CCA challenger. $\mathcal{C}$ generates $\text{pp}_{\text{PKE}} \leftarrow \text{PKE.Setup}(1^\lambda)$ and $(\text{PKE.pk}, \text{PKE.dk}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, and gives $(\text{pp}_{\text{PKE}}, \text{PKE.pk})$ to $\mathcal{B}_1$. $\mathcal{B}_1$ runs $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda)$ and sets $\text{pp} = (\text{pp}_{\text{Sig}}, \text{pp}_{\text{PKE}})$. $\mathcal{B}_1$ chooses $b'' \xleftarrow{\$} \{0, 1\}$ and sets $\text{mpk}_{b''} = \text{PKE.pk}$. $\mathcal{B}_1$ runs $(\text{mpk}_{1-b''}, \text{msk}_{1-b''}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$. $\mathcal{B}_1$ runs $(\text{vk}^*, \text{sigk}^*) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$ and sends $(M_0^*, M_1^*) = (\text{sigk}^*, 0^{|\text{sigk}^*|})$ to $\mathcal{C}$ as the challenge query. $\mathcal{C}$ chooses $b \xleftarrow{\$} \{0, 1\}$, computes the challenge ciphertext $C_{\text{PKE}}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}, M_b^*)$, and returns C_{PKE}^* to $\mathcal{B}_1$. $\mathcal{B}_1$ sets $\text{dpk}^* = (\text{vk}^*, C_{\text{PKE}}^*)$ and gives $(\text{pp}, \text{mpk}_0, \text{mpk}_1, \text{dpk}^*)$ to $\mathcal{A}$. $\mathcal{B}_1$ initializes $L_{\text{dpk},0} := \emptyset$ and $L_{\text{dpk},1} := \emptyset$. $\mathcal{B}_1$ answers queries issued by $\mathcal{A}$ as follows.

Derived Public Key Check Query: $\mathcal{A}$ sends $\text{dpk} \neq \text{dpk}^*$ and index $i \in \{0, 1\}$ to $\mathcal{B}$. Let $\text{dpk} = (\text{vk}, C_{\text{PKE}})$. If $\text{dpk} \neq \text{dpk}^*$ and $C_{\text{PKE}} = C_{\text{PKE}}^*$, $\mathcal{B}_1$ returns 0 if $i \neq b''$. Otherwise, if $i = b''$, then $\mathcal{B}_1$ chooses $m \xleftarrow{\$} \text{MSig}$. If $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}^*, m)) = 1$, then $\mathcal{B}$ updates $L_{\text{dpk},i} \leftarrow L_{\text{dpk},i} \cup \{\text{dpk}\}$ and returns 1. If $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}^*, m)) = 0$, then $\mathcal{B}_1$ returns 0. If $C_{\text{PKE}} \neq C_{\text{PKE}}^*$ and $i = b''$, $\mathcal{B}_1$ sends C_{PKE} to $\mathcal{C}$ as a decryption query. $\mathcal{C}$ returns $\text{sigk} \leftarrow \text{PKE.Dec}(\text{PKE.dk}, C_{\text{PKE}})$ to $\mathcal{B}_1$. $\mathcal{B}_1$ returns 0 if $\text{sigk} = \perp$. Otherwise, $\mathcal{B}_1$ chooses $m \xleftarrow{\$} \text{MSig}$. If $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}, m)) = 1$, then $\mathcal{B}_1$ updates $L_{\text{dpk},i} \leftarrow L_{\text{dpk},i} \cup \{\text{dpk}\}$ and returns 1. If $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}, m)) = 0$, then $\mathcal{B}_1$ returns 0. If $C_{\text{PKE}} \neq C_{\text{PKE}}^*$ and $i \neq b''$, $\mathcal{B}_1$ computes $\text{sigk} \leftarrow \text{PKE.Dec}(\text{msk}_{1-b''}, C_{\text{PKE}})$. $\mathcal{B}_1$ chooses $m \xleftarrow{\$} \text{MSig}$. If $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}, m)) = 1$, then $\mathcal{B}_1$ updates $L_{\text{dpk},i} \leftarrow L_{\text{dpk},i} \cup \{\text{dpk}\}$ and returns 1. If $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}, m)) = 0$, then $\mathcal{B}_1$ returns 0.

Derived Secret Key Corruption Query: $\mathcal{A}$ sends $\text{dpk} \in L_{\text{dpk},0} \cup L_{\text{dpk},1}$ to $\mathcal{B}$. Let $\text{dpk} = (\text{vk}, C_{\text{PKE}})$. If $\text{dpk} \neq \text{dpk}^*$, $C_{\text{PKE}} = C_{\text{PKE}}^*$, $\mathcal{B}_1$ returns sigk^* to $\mathcal{A}$. If $C_{\text{PKE}} \neq C_{\text{PKE}}^*$ and $i = b''$, $\mathcal{B}_1$ sends C_{PKE} to $\mathcal{C}$ as a decryption query. $\mathcal{C}$ returns $\text{sigk} \leftarrow \text{PKE.Dec}(\text{PKE.dk}_i, C_{\text{PKE}})$ to $\mathcal{B}_1$. $\mathcal{B}$

returns sigk to $\mathcal{A}$. If $C_{\text{PKE}} \neq C_{\text{PKE}}^*$ and $i \neq b''$, $\mathcal{B}_1$ computes $\text{sigk} \leftarrow \text{PKE.Dec}(\text{msk}_{1-b''}, C_{\text{PKE}})$ and returns sigk .

Signing Query: $\mathcal{A}$ sends $M \in \text{MS}_{\text{PDPKS}}$ and $\text{dpk} \in L_{\text{dpk},0} \cup L_{\text{dpk},1} \cup \{\text{dpk}^*\}$ to $\mathcal{C}$. Let $\text{dpk} = (\text{vk}, C_{\text{PKE}})$. If $C_{\text{PKE}} = C_{\text{PKE}}^*$ and $i = b''$, then $\mathcal{B}_1$ returns $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}^*, M)$. Otherwise, $\mathcal{B}_1$ retrieves $\text{dsk} = \text{sigk}$ by internally issuing a derived secret key corruption query or uses sigk if $\mathcal{A}$ has issued dpk as a derived secret key corruption query. $\mathcal{B}_1$ returns $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}, M)$.

If $b = 0$, then $\mathcal{B}_1$ simulates Game_0 and if $b = 1$, $\mathcal{B}_1$ simulates Game_1 . Thus, $|\Pr[E_0] - \Pr[E_1]| \leq \text{Adv}_{\mathcal{B}_1, \text{PKE}}^{\text{IND-CCA}}(\lambda)$ holds.

Game₂. This is the same as Game_1 except that the response of derived public key check queries is changed. Let $\text{dpk} = (\text{vk}, C_{\text{PKE}})$ be a derived public key check query. If $\text{vk} \neq \text{vk}^*$, $C_{\text{PKE}} = C_{\text{PKE}}^*$, and $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}^*, m)) = 1$ holds for a random choice of $m \in \text{MS}_{\text{Sig}}$, then the challenger outputs 0 for a derived public key check query. If this event does not happen, Game_1 and Game_2 are identical. Thus, $|\Pr[E_1] - \Pr[E_2]| \leq \Pr[E]$ holds where $\Pr[E]$ is the probability that the event happens. We show that $\Pr[E]$ is negligible if Sig provides the S-CEO security as follows. Let $\mathcal{A}$ be an adversary of unlinkability and $\mathcal{C}$ be the challenger of the signature scheme. We construct an algorithm $\mathcal{B}_2$ that breaks the S-CEO security if the event happens as follows. $\mathcal{C}$ runs $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda)$ and $(\text{vk}^*, \text{sigk}^*) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$ and sends $(\text{pp}_{\text{Sig}}, \text{vk}^*)$ to $\mathcal{B}_2$. $\mathcal{B}_2$ chooses $b'' \xleftarrow{\$} \{0, 1\}$, runs $\text{pp}_{\text{Sig}} \leftarrow \text{Sig.Setup}(1^\lambda)$, $(\text{mpk}_{b''}, \text{msk}_{b''}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, and $(\text{mpk}_{1-b''}, \text{msk}_{1-b''}) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, computes $C_{\text{PKE}}^* \leftarrow \text{PKE.Enc}(\text{mpk}_{b''}, 0^{|\text{sigk}|})$, sets $\text{pp} = (\text{pp}_{\text{Sig}}, \text{pp}_{\text{PKE}})$ and $\text{dpk}^* = (\text{vk}^*, C_{\text{PKE}}^*)$, and gives $(\text{pp}, \text{mpk}_0, \text{mpk}_1, \text{dpk}^*)$ to $\mathcal{A}$. $\mathcal{B}_2$ initializes $L_{\text{dpk},0} := \emptyset$ and $L_{\text{dpk},1} := \emptyset$. $\mathcal{B}_2$ answers queries issued by $\mathcal{A}$ as follows.

Derived Public Key Check Query: $\mathcal{A}$ sends $\text{dpk} \neq \text{dpk}^*$ and index $i \in \{0, 1\}$ to $\mathcal{B}_2$. Let $\text{dpk} = (\text{vk}, C_{\text{PKE}})$. If $\text{dpk} \neq \text{dpk}^*$, $C_{\text{PKE}} = C_{\text{PKE}}^*$, $\mathcal{B}_2$ returns 0 if $i \neq b''$. Otherwise, if $i = b''$, then $\mathcal{B}_2$ chooses $m \xleftarrow{\$} \text{MS}_{\text{Sig}}$ and sends m to $\mathcal{C}$ as a signing query, and obtains Σ . If $\text{Sig.Verify}(\text{vk}, m, \Sigma) = 1$ (i.e., the event happens), then $\mathcal{B}_2$ outputs (vk, m, Σ) that breaks the S-CEO security. If $\text{Sig.Verify}(\text{vk}, m, \Sigma) = 0$, then $\mathcal{B}_2$ returns 0. If $C_{\text{PKE}} \neq C_{\text{PKE}}^*$, $\mathcal{B}_2$ computes $\text{sigk} \leftarrow \text{PKE.Dec}(\text{msk}_i, C_{\text{PKE}})$. $\mathcal{B}_2$ returns 0 if $\text{sigk} = \perp$. Otherwise, $\mathcal{B}_2$ chooses $m \xleftarrow{\$} \text{MS}_{\text{Sig}}$. If $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}, m)) = 1$, then $\mathcal{B}_2$ updates $L_{\text{dpk},i} \leftarrow L_{\text{dpk},i} \cup \{\text{dpk}\}$ and returns 1. If $\text{Sig.Verify}(\text{vk}, m, \text{Sig.Sign}(\text{sigk}, m)) = 0$, then $\mathcal{B}_2$ returns 0.

Derived Secret Key Corruption Query: $\mathcal{A}$ sends $\text{dpk} \in L_{\text{dpk},0} \cup L_{\text{dpk},1}$ to $\mathcal{B}_2$. Let $\text{dpk} = (\text{vk}, C_{\text{PKE}})$. If $\text{dpk} \neq \text{dpk}^*$, $C_{\text{PKE}} = C_{\text{PKE}}^*$, and $i = b''$, $\mathcal{B}_2$ has broken the S-CEO security. Now, $C_{\text{PKE}} \neq C_{\text{PKE}}^*$. $\mathcal{B}_2$ computes $\text{sigk} \leftarrow \text{PKE.Dec}(\text{msk}_i, C_{\text{PKE}})$ and returns sigk .

Signing Query: $\mathcal{A}$ sends $M \in \text{MS}_{\text{PDPKS}}$ and $\text{dpk} \in L_{\text{dpk},0} \cup L_{\text{dpk},1} \cup \{\text{dpk}^*\}$ to $\mathcal{B}_2$. Let $\text{dpk} = (\text{vk}, C_{\text{PKE}})$. If $\text{dpk} = \text{dpk}^*$, then $\mathcal{B}_2$ returns $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}^*, M)$. Otherwise, $\mathcal{B}_2$ retrieves $\text{dsk} = \text{sigk}$ by internally issuing a derived secret key corruption query or uses sigk if $\mathcal{A}$ has issued dpk as a derived secret key corruption query. $\mathcal{B}_2$ returns $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}, M)$.

If the event happens, then $\mathcal{B}_2$ breaks the S-CEO security. Thus, $\Pr[E] \leq \text{Adv}_{\mathcal{B}_2, \text{Sig}}^{\text{S-CEO}}(\lambda)$ holds.

Finally, in Game_2 , we construct an algorithm $\mathcal{B}_3$ that breaks key privacy, i.e., $|\Pr[E_2] - 1/2| \leq \text{Adv}_{\mathcal{B}_3, \text{PKE}}^{\text{Key-Privacy}}(\lambda)$, as follows. Let $\mathcal{A}$ be an adversary and $\mathcal{C}$ be the challenger of the PKE scheme. $\mathcal{C}$ generates $\text{pp}_{\text{PKE}} \leftarrow \text{PKE.Setup}(1^\lambda)$, $(\text{PKE.pk}_0, \text{PKE.dk}_0) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, and $(\text{PKE.pk}_1, \text{PKE.dk}_1) \leftarrow \text{PKE.KeyGen}(\text{pp}_{\text{PKE}})$, and gives $(\text{pp}_{\text{PKE}}, \text{PKE.pk}_0, \text{PKE.pk}_1)$ to $\mathcal{B}_3$. $\mathcal{B}_3$ runs $\text{pp}_{\text{Sig}} \leftarrow$

$\text{Sig.Setup}(1^\lambda)$ and $(vk^*, \text{sigk}^*) \leftarrow \text{Sig.KeyGen}(\text{pp}_{\text{Sig}})$. $\mathcal{B}_3$ sends $0^{|\text{sigk}^*|}$ to $\mathcal{C}$ as the challenge plaintext. $\mathcal{C}$ chooses $b \xleftarrow{\$} \{0, 1\}$, computes the challenge ciphertext $C_{\text{PKE}}^* \leftarrow \text{PKE.Enc}(\text{PKE.pk}_b, 0^{|\text{sigk}^*|})$, and gives C_{PKE}^* to $\mathcal{B}_3$. $\mathcal{B}_3$ sets $\text{pp} = (\text{pp}_{\text{Sig}}, \text{pp}_{\text{PKE}})$, $(\text{mpk}_0, \text{mpk}_1) = (\text{PKE.pk}_0, \text{PKE.pk}_1)$, and $\text{dpk}^* = (vk^*, C_{\text{PKE}}^*)$, and sends $(\text{pp}, \text{mpk}_0, \text{mpk}_1, \text{dpk}^*)$ to $\mathcal{A}$. $\mathcal{B}_3$ initializes $L_{\text{dpk},0} := \emptyset$ and $L_{\text{dpk},1} := \emptyset$. $\mathcal{B}_3$ answers queries issued by $\mathcal{A}$ as follows.

Derived Public Key Check Query: $\mathcal{A}$ sends $\text{dpk} \neq \text{dpk}^*$ and index $i \in \{0, 1\}$ to $\mathcal{B}_3$. Let $\text{dpk} = (vk, C_{\text{PKE}})$. If $\text{dpk} \neq \text{dpk}^*$ and $C_{\text{PKE}} = C_{\text{PKE}}^*$, then $\mathcal{B}_3$ returns 0 regardless of i due to the modification of the previous game. If $C_{\text{PKE}} \neq C_{\text{PKE}}^*$, $\mathcal{B}_3$ sends (C_{PKE}, i) to $\mathcal{C}$ as a decryption query where $\text{dpk} \in L_{\text{dpk},i}$. $\mathcal{C}$ returns $\text{sigk} \leftarrow \text{PKE.Dec}(\text{PKE.dk}_i, C_{\text{PKE}})$ to $\mathcal{B}_3$. $\mathcal{B}_3$ returns 0 if $\text{sigk} = \perp$. Otherwise, $\mathcal{B}_3$ chooses $m \xleftarrow{\$} \text{MS}_{\text{Sig}}$. If $\text{Sig.Verify}(vk, m, \text{Sig.Sign}(\text{sigk}, m)) = 1$, then $\mathcal{B}_3$ updates $L_{\text{dpk},i} \leftarrow L_{\text{dpk},i} \cup \{\text{dpk}\}$ and returns 1. If $\text{Sig.Verify}(vk, m, \text{Sig.Sign}(\text{sigk}, m)) = 0$, then $\mathcal{B}_3$ returns 0.

Derived Secret Key Corruption Query: $\mathcal{A}$ sends $\text{dpk} \in L_{\text{dpk},0} \cup L_{\text{dpk},1}$ to $\mathcal{B}_3$. Let $\text{dpk} = (vk, C_{\text{PKE}})$. Note that if $C_{\text{PKE}} = C_{\text{PKE}}^*$ (i.e., $vk \neq vk^*$), then $\text{dpk} \notin L_{\text{dpk},0} \cup L_{\text{dpk},1}$. Thus, we consider the case that $C_{\text{PKE}} \neq C_{\text{PKE}}^*$. $\mathcal{B}_3$ sends (C_{PKE}, i) to $\mathcal{C}$ as a decryption query where $\text{dpk} \in L_{\text{dpk},i}$. $\mathcal{C}$ returns $\text{sigk} \leftarrow \text{PKE.Dec}(\text{PKE.dk}_i, C_{\text{PKE}})$ to $\mathcal{B}_3$. $\mathcal{B}_3$ returns sigk to $\mathcal{A}$.

Signing Query: $\mathcal{A}$ sends $M \in \text{MS}_{\text{PDPKS}}$ and $\text{dpk} \in L_{\text{dpk},0} \cup L_{\text{dpk},1} \cup \{\text{dpk}^*\}$ to $\mathcal{C}$. If $\text{dpk} = \text{dpk}^*$, then $\mathcal{B}_3$ returns $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}^*, M)$. Otherwise, $\mathcal{B}_3$ retrieves $\text{dsk} = \text{sigk}$ by internally issuing a derived secret key corruption query or uses sigk if $\mathcal{A}$ has issued dpk as a derived secret key corruption query. $\mathcal{B}_3$ returns $\Sigma \leftarrow \text{Sig.Sign}(\text{sigk}, M)$.

Finally, $\mathcal{A}$ outputs $b' \in \{0, 1\}$. $\mathcal{B}_3$ outputs b' and breaks key privacy with the advantage at least $\text{Adv}_{\mathcal{A}, \text{PDPKS}}^{\text{unlink}}(\lambda)$. Thus, $|\Pr[E_2] - 1/2| \leq \text{Adv}_{\mathcal{B}_3, \text{PKE}}^{\text{Key-Privacy}}(\lambda)$ holds.

Now, we have

$$\begin{aligned}
\text{Adv}_{\mathcal{A}, \text{PDPKS}}^{\text{unlink}}(\lambda) &= |\Pr[E_0] - 1/2| \\
&= |\Pr[E_0] - \Pr[E_1] + \Pr[E_1] - \Pr[E_2] + \Pr[E_2] - 1/2| \\
&\leq |\Pr[E_0] - \Pr[E_1]| + |\Pr[E_1] - \Pr[E_2]| + |\Pr[E_2] - 1/2| \\
&\leq |\Pr[E_0] - \Pr[E_1]| + \Pr[E] + |\Pr[E_2] - 1/2| \\
&\leq \text{Adv}_{\mathcal{B}_1, \text{PKE}}^{\text{IND-CCA}}(\lambda) + \text{Adv}_{\mathcal{B}_2, \text{Sig}}^{\text{S-CEO}}(\lambda) + \text{Adv}_{\mathcal{B}_3, \text{PKE}}^{\text{Key-Privacy}}(\lambda)
\end{aligned}$$

This concludes the proof. $\square$

6 One-time PDPKS

Liu et al. [37] mentioned that “Note that the concept of PDPKS is motivated by the security and privacy problems in cryptocurrency, where it is suggested that each public/verification key, as the coin address, is used only once. But in this paper we do not restrict the concept to one-time signature scheme, which requires that for each public key the signing oracle can be queried at most once. Our proposed PDPKS requires stronger security, namely, even if the users use the freshly derived key pairs multiple times, the system is still safe.”. Thus, in their security model (and our outsider strong unforgeability), an adversary $\mathcal{A}$ is allowed to issue signing queries (dpk^*, M) in multiple times.

Again, each $\mathbf{dpk}$ is used only once as a coin-receiving address, and each fresh $\mathbf{dpk}$ is a different value (if a different randomness is used for key derivation). From this perspective, we can define a one-time variant of outsider strong unforgeability where $\mathcal{A}$ is allowed to issue a signing query $(\mathbf{dpk}^*, M)$ only once. Intuitively, we can employ a one-time signature scheme instead of a signature scheme that seems effective to improve the efficiency of PDPKS schemes instantiated via the proposed generic construction. For example, with the Cremers et al.’s conversion [21] to add the S-CEO security, we may be able to employ the DL based Groth strongly unforgeable one-time signature scheme [30, 48] or the lattice-based Lyubashevsky-Micciancio strongly unforgeable one-time signature scheme [39]. However, in the security proof of outsider strong unforgeability, the reduction algorithm $\mathcal{B}_2$ may issue signing queries to the challenger more than once. Concretely, let $\mathcal{A}$ issue $\mathbf{dpk} = (\mathbf{vk}, C_{\text{PKE}}^*)$ where $\mathbf{vk} \neq \mathbf{vk}^*$ as a derived public key check query (resp. a derived secret key key query). Then, $\mathcal{B}_2$ sends m to $\mathcal{C}$ as a signing query and obtains Σ . If $\text{Sig.Verify}(\mathbf{vk}, m, \Sigma) = 1$ holds, then $\mathcal{B}_2$ breaks the S-CEO security. However, if $\text{Sig.Verify}(\mathbf{vk}, m, \Sigma) = 0$, then $\mathcal{B}_2$ returns 0 (resp. $\perp$) to $\mathcal{A}$ and the game goes on. Later, if $\mathcal{A}$ issues a signing query $(\mathbf{dpk}^*, M)$, then $\mathcal{B}_2$ needs to send a signing query M to $\mathcal{C}$ but it is prohibited if the signature scheme is restricted as one-time use. The same thing happens in the proof of unlinkability. Thus, though one-time PDPKS seems sufficient for cryptocurrency applications such as stealth address and deterministic wallet, our generic construction is currently not applicable for instantiating a one-time PDPKS scheme. We leave how to construct an efficient one-time PDPKS scheme is left as a future work of this paper.

7 Conclusion

In this paper, we introduced consistency and outsider strong unforgeability, and proposed a generic construction of PDPKS from signatures and PKE. To the best of our knowledge, no isogeny-based CCA secure key private PKE scheme has been proposed so far. Thus, currently our generic construction does not give an isogeny-based PDPKS scheme. Das et al. [23] proposed an isogeny-based deterministic threshold wallet. Though the functionality is incompatible to PDPKS, their construction technique may be applicable to PDPKS. We leave it as a future work of this paper. Zhu et al. [49] investigated universal composability (UC) of PDPKS and showed that the game-based definitions (unforgeability and unlinkability) given by Liu et al. [37] and the UC-security of PDPKS are equivalent. Since we gave a new security definition of PDPKS, it would be desirable to investigate whether our definition is still equivalent to the UC-security or not. We leave it as a future work of this paper.

Acknowledgment: This work was supported by JSPS KAKENHI Grant Numbers JP21K11897, JP23K24844, and JP25H01106.

References

- [1] Michel Abdalla, Mihir Bellare, and Gregory Neven. Robust encryption. In Daniele Micciancio, editor, *TCC 2010*, volume 5978 of *LNCS*, pages 480–497. Springer, Berlin, Heidelberg, February 2010.
- [2] Michel Abdalla, Mihir Bellare, and Phillip Rogaway. The oracle Diffie-Hellman assumptions and an analysis of DHIES. In David Naccache, editor, *CT-RSA 2001*, volume 2020 of *LNCS*, pages 143–158. Springer, Berlin, Heidelberg, April 2001.

- [3] Shweta Agrawal, Dan Boneh, and Xavier Boyen. Efficient lattice (H)IBE in the standard model. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 553–572. Springer, Berlin, Heidelberg, May / June 2010.
- [4] Nabil Alkeilani Alkadri, Poulami Das, Andreas Erwig, Sebastian Faust, Juliane Krämer, Siavash Riahi, and Patrick Struck. Deterministic wallets in a quantum world. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1017–1031. ACM Press, November 2020.
- [5] Thomas Aulbach, Samed Düzl , Michael Meyer, Patrick Struck, and Maximiliane Weish upl. Hash your keys before signing - BUFF security of the additional NIST PQC signatures. In Markku-Juhani Saarinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part II*, pages 301–335. Springer, Cham, June 2024.
- [6] Lukas Aumayr, Oguzhan Ersoy, Andreas Erwig, Sebastian Faust, Kristina Host kov , Matteo Maffei, Pedro Moreno-Sanchez, and Siavash Riahi. Generalized channels from limited blockchain scripts and adaptor signatures. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part II*, volume 13091 of *LNCS*, pages 635–664. Springer, Cham, December 2021.
- [7] Paulo S. L. M. Barreto, Beno t Libert, Noel McCullagh, and Jean-Jacques Quisquater. Efficient and provably-secure identity-based signatures and signcryption from bilinear maps. In Bimal K. Roy, editor, *ASIACRYPT 2005*, volume 3788 of *LNCS*, pages 515–532. Springer, Berlin, Heidelberg, December 2005.
- [8] Paulo S. L. M. Barreto, Ben Lynn, and Michael Scott. Constructing elliptic curves with prescribed embedding degrees. In Stelvio Cimato, Clemente Galdi, and Giuseppe Persiano, editors, *SCN 02*, volume 2576 of *LNCS*, pages 257–267. Springer, Berlin, Heidelberg, September 2003.
- [9] Paulo S. L. M. Barreto and Michael Naehrig. Pairing-friendly elliptic curves of prime order. In Bart Preneel and Stafford Tavares, editors, *SAC 2005*, volume 3897 of *LNCS*, pages 319–331. Springer, Berlin, Heidelberg, August 2006.
- [10] Mihir Bellare, Alexandra Boldyreva, Anand Desai, and David Pointcheval. Key-privacy in public-key encryption. In Colin Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 566–582. Springer, Berlin, Heidelberg, December 2001.
- [11] Daniel J. Bernstein. Curve25519: New Diffie-Hellman speed records. In Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin, editors, *PKC 2006*, volume 3958 of *LNCS*, pages 207–228. Springer, Berlin, Heidelberg, April 2006.
- [12] Dan Boneh, Aditi Partap, and Brent Waters. Accountable multi-signatures with constant size public keys. In Tibor Jager and Jiaxin Pan, editors, *PKC 2025, Part II*, volume 15675 of *LNCS*, pages 32–65. Springer, Cham, May 2025.
- [13] Dan Boneh, Emily Shen, and Brent Waters. Strongly unforgeable signatures based on computational Diffie-Hellman. In Moti Yung, Yevgeniy Dodis, Aggelos Kiayias, and Tal Malkin, editors, *PKC 2006*, volume 3958 of *LNCS*, pages 229–240. Springer, Berlin, Heidelberg, April 2006.

- [14] Jacqueline Brendel, Cas Cremers, Dennis Jackson, and Mang Zhao. The provable security of Ed25519: Theory and practice. In *2021 IEEE Symposium on Security and Privacy*, pages 1659–1676. IEEE Computer Society Press, May 2021.
- [15] Ran Canetti, Oded Goldreich, and Shai Halevi. The random oracle methodology, revisited. *Journal of the ACM*, 51(4):557–594, 2004.
- [16] Ran Canetti, Shai Halevi, and Jonathan Katz. Chosen-ciphertext security from identity-based encryption. In Christian Cachin and Jan Camenisch, editors, *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 207–222. Springer, Berlin, Heidelberg, May 2004.
- [17] Jung Hee Cheon. Security analysis of the strong Diffie-Hellman problem. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 1–11. Springer, Berlin, Heidelberg, May / June 2006.
- [18] Michele Ciampi, Xiangyu Liu, Ioannis Tzannetos, and Vassilis Zikas. Universal adaptor signatures from blackbox multi-party computation. In Arpita Patra, editor, *CT-RSA 2025*, volume 15598 of *LNCS*, pages 375–398. Springer, April 2025.
- [19] Nicolas T. Courtois and Rebekah Mercer. Stealth address and key management techniques in blockchain systems. In Paolo Mori, Steven Furnell, and Olivier Camp, editors, *ICISSP 2017*, pages 559–566. SciTePress, February 2017.
- [20] Ronald Cramer and Victor Shoup. A practical public key cryptosystem provably secure against adaptive chosen ciphertext attack. In Hugo Krawczyk, editor, *CRYPTO’98*, volume 1462 of *LNCS*, pages 13–25. Springer, Berlin, Heidelberg, August 1998.
- [21] Cas Cremers, Samed Düzl , Rune Fiedler, Marc Fischlin, and Christian Janson. BUFFing signature schemes beyond unforgeability and the case of post-quantum signatures. In *2021 IEEE Symposium on Security and Privacy*, pages 1696–1714. IEEE Computer Society Press, May 2021.
- [22] Wei Dai, Tatsuaki Okamoto, and Go Yamamoto. Stronger security and generic constructions for adaptor signatures. In Takanori Isobe and Santanu Sarkar, editors, *INDOCRYPT 2022*, volume 13774 of *LNCS*, pages 52–77. Springer, Cham, December 2022.
- [23] Poulami Das, Andreas Erwig, Michael Meyer, and Patrick Struck. Efficient post-quantum secure deterministic threshold wallets from isogenies. In Jianying Zhou, Tony Q. S. Quek, Debin Gao, and Alvaro A. C ardenas, editors, *ASIACCS 24*. ACM Press, July 2024.
- [24] Poulami Das, Sebastian Faust, and Julian Loss. A formal treatment of deterministic wallets. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 651–668. ACM Press, November 2019.
- [25] Lo c Ferreira and Johan Pascal. Post-quantum secure ZRTP. In Markku-Juhani Saarinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 15th International Workshop, PQCrypto 2024, Part I*, pages 3–36. Springer, Cham, June 2024.
- [26] Eduarda S. V. Freire, Dennis Hofheinz, Eike Kiltz, and Kenneth G. Paterson. Non-interactive key exchange. In Kaoru Kurosawa and Goichiro Hanaoka, editors, *PKC 2013*, volume 7778 of *LNCS*, pages 254–271. Springer, Berlin, Heidelberg, February / March 2013.

- [27] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 537–554. Springer, Berlin, Heidelberg, August 1999.
- [28] Paul Gerhart, Dominique Schröder, Pratik Soni, and Sri Aravinda Krishnan Thyagarajan. Foundations of adaptor signatures. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part II*, volume 14652 of *LNCS*, pages 161–189. Springer, Cham, May 2024.
- [29] Huseyin Gokay, Foteini Baldimtsi, and Giuseppe Ateniese. Atomic swaps for boneh-lynn-shacham (BLS) based blockchains. In Joaquin Garcia-Alfaro, Rafał Kozik, Michał Choraś, and Sokratis Katsikas, editors, *ESORICS 2024, Part III*, volume 14984 of *LNCS*, pages 341–361. Springer, Cham, September 2024.
- [30] Jens Groth. Simulation-sound NIZK proofs for a practical language and constant size group signatures. In Xuejia Lai and Kefei Chen, editors, *ASIACRYPT 2006*, volume 4284 of *LNCS*, pages 444–459. Springer, Berlin, Heidelberg, December 2006.
- [31] Felix Günther and Marc Ilunga Tshibumbu Mukendi. Careful with MAC-then-SIGN: A computational analysis of the EDHOC lightweight authenticated key exchange protocol. In *2023 IEEE European Symposium on Security and Privacy*, pages 773–796. IEEE Computer Society Press, July 2023.
- [32] Jingwei Jiang and Ding Wang. QPASE: quantum-resistant password-authenticated searchable encryption for cloud storage. *IEEE Transactions on Information Forensics and Security*, 19:4231–4246, 2024.
- [33] Akinori Kawachi, Keisuke Tanaka, and Keita Xagawa. Concurrently secure identification schemes based on the worst-case hardness of lattice problems. In Josef Pieprzyk, editor, *ASIACRYPT 2008*, volume 5350 of *LNCS*, pages 372–389. Springer, Berlin, Heidelberg, December 2008.
- [34] Mukul Kulkarni and Keita Xagawa. Strong existential unforgeability and more of MPC-in-the-head signatures. Cryptology ePrint Archive, Report 2024/1069, 2024.
- [35] Wenling Liu, Zhen Liu, Khoa Nguyen, Guomin Yang, and Yu Yu. A lattice-based key-insulated and privacy-preserving signature scheme with publicly derived public key. In Liqun Chen, Ninghui Li, Kaitai Liang, and Steve A. Schneider, editors, *ESORICS 2020, Part II*, volume 12309 of *LNCS*, pages 357–377. Springer, Cham, September 2020.
- [36] Xiangyu Liu, Ioannis Tzannetos, and Vassilis Zikas. Adaptor signatures: New security definition and a generic construction for NP relations. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part II*, volume 15485 of *LNCS*, pages 168–193. Springer, Singapore, December 2024.
- [37] Zhen Liu, Guomin Yang, Duncan S. Wong, Khoa Nguyen, and Huaxiong Wang. Key-insulated and privacy-preserving signature scheme with publicly derived public key. In *2019 IEEE European Symposium on Security and Privacy*, pages 215–230. IEEE Computer Society Press, June 2019.
- [38] Zhen Liu, Guomin Yang, Duncan S. Wong, Khoa Nguyen, Huaxiong Wang, Xiaorong Ke, and Yining Liu. Secure deterministic wallet and stealth address: Key-insulated and privacy-preserving signature scheme with publicly derived public key. *IEEE Transactions on Dependable and Secure Computing*, 19(5):2934–2951, 2022.

- [39] Vadim Lyubashevsky and Daniele Micciancio. Asymptotically efficient lattice-based digital signatures. In Ran Canetti, editor, *TCC 2008*, volume 4948 of *LNCS*, pages 37–54. Springer, Berlin, Heidelberg, March 2008.
- [40] Alberto Maria Mongardini, Daniele Friolo, and Giuseppe Ateniese. Stealth and beyond: Attribute-driven accountability in bitcoin transactions. Cryptology ePrint Archive, Report 2024/1789, 2024.
- [41] Andrew Poelstra. Scriptless scripts. <https://tinyurl.com/ludcxyz>, 2017.
- [42] Sihang Pu, Sri Aravinda Krishnan Thyagarajan, Nico Döttling, and Lucjan Hanzlik. Post quantum fuzzy stealth signatures and applications. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *ACM CCS 2023*, pages 371–385. ACM Press, November 2023.
- [43] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th ACM STOC*, pages 84–93. ACM Press, May 2005.
- [44] Ron Steinfeld, Josef Pieprzyk, and Huaxiong Wang. How to strengthen any weakly unforgeable signature into a strongly unforgeable signature. In Masayuki Abe, editor, *CT-RSA 2007*, volume 4377 of *LNCS*, pages 357–371. Springer, Berlin, Heidelberg, February 2007.
- [45] Isamu Teranishi, Takuro Oyama, and Wakaha Ogata. General conversion for obtaining strongly existentially unforgeable signatures. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, 91-A(1):94–106, 2008.
- [46] Anton Wahrstätter, Matthew Solomon, Ben DiFrancesco, Vitalik Buterin, and Davor Svetinovic. BaseSAP: Modular stealth address protocol for programmable blockchains. *IEEE Transactions on Information Forensics and Security*, 19:3539–3553, 2024.
- [47] Ruida Wang, Ziyi Li, Xianhui Lu, Zhenfei Zhang, and Kunpeng Wang. Key derivable signature and its application in blockchain stealth address. *Cybersecurity*, 7(1):43, 2024.
- [48] Hoeteck Wee. Public key encryption against related key attacks. In Marc Fischlin, Johannes Buchmann, and Mark Manulis, editors, *PKC 2012*, volume 7293 of *LNCS*, pages 262–279. Springer, Berlin, Heidelberg, May 2012.
- [49] Chunping Zhu, Xingkai Wang, and Zhen Liu. Universally composable key-insulated and privacy-preserving signature scheme with publicly derived public key. In Chunpeng Ge and Moti Yung, editors, *Inscrypt 2023*, pages 44–64, December 2023.

Appendix

In this Appendix, we introduce the construction methodology of Liu et al. [37] which may help the reader to understand their scheme. They pointed out that IBS is a promising tool to construct PDPKS but required a special property, that they called MPK-pack-able property. Briefly, it requires that there exists a function F and a verification algorithm Verify_F , where, for an identity ID and a message-signature pair (M, σ) ,

$$\text{Verify}_F(F(\text{IBS.mpk}, ID), M, \sigma) = \text{IBS.Verify}(\text{IBS.mpk}, ID, M, \sigma)$$

holds and no information of IBS.mpk is leaked from $F(\text{IBS.mpk}, ID)$. The main reason why Liu et al. introduced the MPK-pack-able property is the verification algorithm of IBS needs to take the corresponding master public key that violates the unlinkability. Intuitively, for $(\text{IBS.mpk}_B, \text{IBS.msk}_B) \leftarrow \text{IBS.MasterKeyGen}(\text{pp}_{\text{IBS}})$, where pp_{IBS} is a common parameter, a payee Bob sets $\text{mpk}_B = \text{IBS.mpk}_B$ and $\text{msk}_B = \text{IBS.msk}_B$. A payer Alice computes $F(\text{mpk}, ID)$ for some ID , and sets $\text{dpk}_A = F(\text{mpk}_B, ID)$. Bob generates $\text{IBS.sk} \leftarrow \text{IBS.KeyDer}(\text{mpk}_B, \text{msk}_B, ID)$ and sets $\text{dsk}_B = \text{IBS.sk}$. Then, due to the MPK-pack-able property, an IBS signature σ generated by dsk_B can be verified by $\text{dpk}_A = F(\text{mpk}_B, ID)$ and information of mpk_B is not leaked from dpk_A .

The last piece is how Bob to know ID and how Bob to check the validity of dpk_A . Liu et al. implicitly employed non-interactive key exchange (NIKE) [26].¹⁷ By using a NIKE protocol, the construction idea of Liu et al. is explained as follows. A payee Bob sets $\text{mpk}_B = (\text{IBS.mpk}_B, \text{NIKE.pk}_B)$ and $\text{msk}_B = (\text{IBS.msk}_B, \text{NIKE.sk}_B)$. A payer Alice generates a fresh key pair $(\text{NIKE.pk}_A, \text{NIKE.sk}_A)$ and computes a shared key shk_{AB} by $(\text{NIKE.sk}_A, \text{NIKE.pk}_B)$. Let $F = (\text{Commit}, \text{ComOpen})$ be a commitment scheme. Alice sets

$$\text{dpk}_A = (c, \text{NIKE.pk}_A)$$

where $c = \text{Commit}(\text{mpk}_B, \text{shk}_{AB})$, i.e., the shared key shk_{AB} is set as the decommit (and ID in the above explanation). Due to the hiding property of the commitment scheme, no information of mpk_B is leaked from c . Moreover, NIKE.pk_A is independent of mpk_B . Due to the binding property of the commitment scheme, mpk_B is linked to c . Bob also generates shk_{AB} by $(\text{NIKE.pk}_A, \text{NIKE.sk}_B)$ and checks whether c is a commitment of mpk_B by $\text{ComOpen}(\text{mpk}_B, \text{shk}_{AB}, c)$. To support the condition $\text{Verify}_F(\text{Commit}(\text{mpk}_B, \text{shk}_{AB}), M, \sigma) = \text{IBS.Verify}(\text{IBS.mpk}_B, \text{shk}_{AB}, M, \sigma)$, the MPK-pack-able property has the central roles of this construction methodology.

¹⁷Let two users, Alice and Bob, would like to share a key. Then, Alice and Bob generate $(\text{NIKE.pk}_A, \text{NIKE.sk}_A)$ and $(\text{NIKE.pk}_B, \text{NIKE.sk}_B)$, respectively. Then, a shared key shk_{AB} can be generated by either $(\text{NIKE.pk}_A, \text{NIKE.sk}_B)$ or $(\text{NIKE.sk}_A, \text{NIKE.pk}_B)$ and is indistinguishable from random. In the actual syntax of NIKE, identities (Alice and Bob here) are also included to generate shk_{AB} . We omit them here.